

第二章 进程的描述与控制

2025年4月29日 10:15

$$P = \{P_1, P_2, P_3, P_4, P_5, P_6, P_7, P_8, P_9\}$$
$$= \{(P_1, P_2), (P_1, P_3), (P_1, P_4), (P_2, P_5), (P_3, P_5), (P_4, P_6), (P_4, P_7), (P_5, P_8), (P_6, P_8), (P_7, P_9), (P_8, P_9)\}$$

早期未配置的OS系统和单道批处理系统中程序的执行方式是**顺序执行**，内存中只装入一道用户程序，浪费资源，系统运行效率低

为描述程序执行顺序和并发执行情况，引入**前趋图**， $P_1 \rightarrow P_2$ 或者前趋图不允许有循环

程序顺序执行的三个特征：

- 顺序性：指处理机严格地按照程序所规定的顺序执行
- 封闭性：程序运行所需的资源只有本程序才能改变
- 可再现性：环境和初始条件不变，程序执行结果相同

程序并发执行（资源共享）的三个特征：

- 间断性；失去封闭性；不可再现性

进程描述：

进程的定义：

- 是程序的一次执行
- 是一个程序及其数据在处理机上顺序执行时所发生的活动
- 是具有独立功能的程序在一个数据集合上运行的过程
- 是系统进行资源分配和调度的一个独立单位

进程的特征：

- 进程具有程序所没有的PCB结构
- 动态性、并发性、独立性、异步性

进程的三种基本状态：就绪、执行、阻塞

- 就绪——调度程序分配处理机——》执行
- 执行——分配的时间片已经用完从而被剥离处理机暂停执行——》就绪
- 执行——进程访问临界资源而该资源当前不可访问、执行受阻——》阻塞

程序还可能在运行阶段被Sleep或Suspend进入睡眠或挂起状态，之后在睡眠结束或Resume后会进入就绪状态

创建状态与终止状态

创建进程：

- 1.进程申请空白PCB，并向PCB中填写用于控制和管理进程的信息
 - 2.为该进程分配运行时所必需的资源
 - 3.初始化进程控制块(PCB，如标识、状态和控制信息
 - 4.把该进程转入就绪状态，插入就绪队列中
- 如果进程所需资源不能得到满足（包括正在创建中），则称此时所处状态为**创建状态**

终止进程：

- 1.等待操作系统进行善后处理
 - 2.将PCB清零，将空间返还系统
- 进入终止状态的原因：到达自然结束点、遇到不可恢复的错误、被操作系统终止，或被其他有终止权限的进程终止
- 处于终止状态的进程不再执行，但是操作系统会暂时保留其记录，保存**状态码及计时统计数据**，供其他进程收集。当其他进程提取完必要信息后，操作系统会彻底删除该进程，即将其PCB**清空并回收**

挂起操作：

- 挂起原因：终端用户、负荷调节、操作系统的需要，父进程请求
- 活动就绪→静止就绪。
- 活动阻塞→静止阻塞。
- 静止就绪→活动就绪。
- 静止阻塞→活动阻塞。

进程管理的数据结构

- 对每个资源和每个进程都设置一个数据结构来表征实体
- 数据结构包含资源或进程的标识、描述、状态等信息以及一批指针
- 通过指针可以将资源分类链接从而便于系统查找
- OS管理的数据结构一般分为内存表、设备表、文件表和用于进程管理的进程表（进程控制块）

PCB的作用

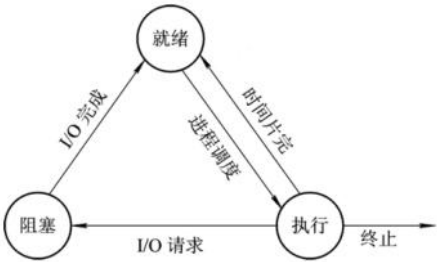
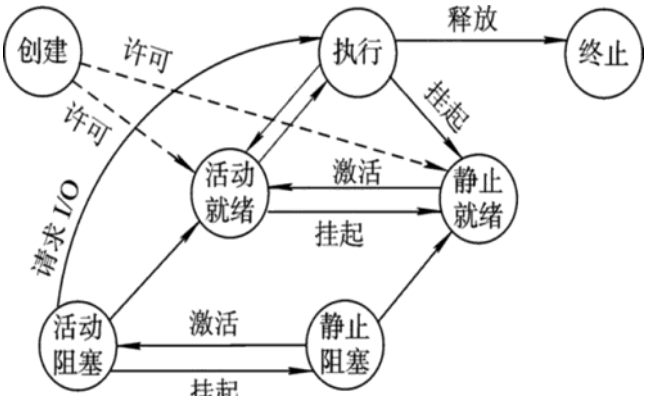
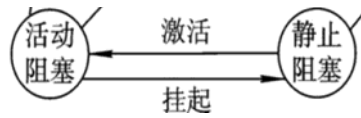


图2-5 进程的三种基本状态及其转换



OS管理的数据结构一般分为内存表、设备表、文件表和用于进程管理的进程表（进程控制块）



PCB的作用

- 1.作为独立运行基本单位的标志：随进程创建、消亡而产生、消亡
- 2.可以实现间断性运行方式：PCB保存和恢复现场信息
- 3.提供了进程管理所需要的信息：资源清单、程序、数据
- 4.提供进程调度所需信息：进程状态等
- 5.实现与其他进程的同步与通信：提供信号量、队列指针

PCB中的信息：

进程标识符：唯一的标识一个进程，一般分为外部标识符（创建者提供，数字字母组成）和内部标识符（数字标识，OS使用）

处理机状态：由各种寄存器的内容组成，寄存器包括：通用寄存器、指令寄存器、程序状态字、用户栈指针

进程调度信息：OS进行调度时所必须了解的内容，包括：进程状态（进程调度和对换时的依据）、进程优先级（整数，优先级越高越优先获得处理机）、进程调度所需的其他信息（与进程调度算法有关，比如进程已等待时间总和、已执行时间总和等）、事件（进程执行状态所需发生的事件/原因）

进程控制信息：进程控制所必须的信息，包括：程序和数据地址（进程实体中程序和数据内存或外存（首）地址，方便查找）、进程同步和通信机制（消息队列指针、信号量等，进程同步和通信所必须的机制，全部或部分的放在PCB中）、资源清单（运行期间所需的除了CPU以外的全部资源）、已分配到该进程的资源清单、链接指针（本进程所在队列的下一个进程的首地址）

PCB的组织方式：

线性方式

组织在线性表中（将该表的首址存放在内存的一个专用区域中），实现简单开销小，查找消耗大，适合进程数目不多的系统

链接方式

同状态进程组织成一个队列，包括就绪队列（可按照优先级高低组织）、执行队列、阻塞队列（可以根据阻塞原因再细分）、空闲队列

索引方式

建立就绪索引表、阻塞索引表等（把各索引表在内存的首地址记录在内存的一些专用单元中），每个索引表的表目记录着PCB在PCB表中的地址

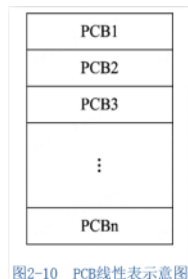


图2-10 PCB线性表示意图

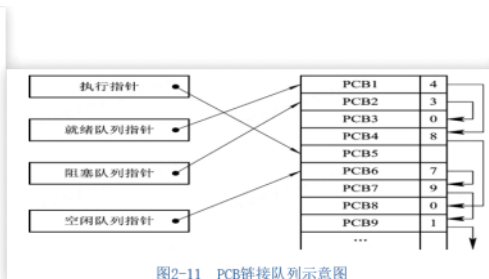


图2-11 PCB链接队列示意图

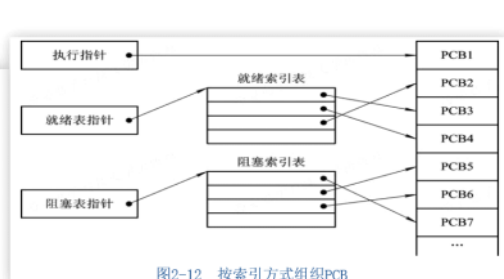


图2-12 按索引方式组织PCB

进程控制：

进程控制时进程管理中最基本的功能，包括创建新进程、终止已经完成的进程、置进程于阻塞状态、转换转换运行中的进程状态、进程控制一般由OS内核中的**原语**来实现

OS内核：将一些与硬件紧密相关的模块（中断处理程序等）、各种常用的设备驱动程序以及运行频率较高的模块（时钟管理、进程调度等），都安排在紧靠硬件的层次中，将它们常驻内存，即通常被称为OS内核

OS内核的两大功能

支撑功能

中断处理:支撑并发性的实现

时钟管理:支撑时间片轮转等其他任务活动

原语操作:不可中断的操作序列

资源管理功能

进行管理:创建撤销、调度进程

存储器管理: 地址分配回收、地址映射、调页

设备管理: 设备驱动、缓冲和分配等

层次结构：

unix中进程具有层次结构，允许一个进程创建另一个进程，构成父子进程关系，子进程继承父进程的资源，子进程撤销时，将资源还给父进程，撤销父进程应先撤销子进程

windows系统中没有层次关系，通过进程的句柄控制，拥有句柄就可以控制进程

进程的创建

引起进程创建的事件：用户登录、作业调度、提供服务、应用请求

进程的创建流程（系统出现创建新进程的请求后，OS调用**Creat**原语创建进程）

- 1.申请空白PCB：为新进程申请获得唯一的数字标识符，并从PCB集合中索取一个空白PCB。
- 2.为新进程分配运行所需资源：包括物理（内存、文件、IO设备）和逻辑资源（cpu时间）
- 3.初始化进程控制块(PCB，如标识、状态和控制信息)
- 4.如果进程就绪队列能够接纳新进程，便将新进程插入就绪队列。

进程的终止

引起进程终止的事件：正常结束、异常结束（越界、保护错、非法指令、运行或等待超时等）、外界干预（用户或系统请求、父进程请求、父进程被撤销）

进程的终止流程（系统中发生了要求终止进程的某事件，OS便调用进程**终止**原语）

- 1.根据被终止进程的标识符，从PCB集合中检索出该进程的PCB，读取该进程的状态
- 2.若被终止进程正在执行，则立即终止该进程的执行，并置调度标志位真，用于指示该进程被终止后应重新进行调度
- 3.若该进程还有子孙进程，还应将子孙进程也予以重视，以防他们成为不可控的进程
- 4.将被终止进程所拥有的全部资源归还给父进程或系统
- 5.将被终止进程从所在队列或链表中移出，等待其他程序来搜集信息

进程的阻塞与唤醒

引起进程阻塞和唤醒的事件：向系统请求共享资源失败/成功、等待某种操作的完成、新数据尚未到达、等待新任务的到达

进程阻塞过程（进程便通过调用阻塞原语**block**将自己阻塞）

阻塞时进程自身的主动行为，进入block过程后，将该进程立即停止执行，将PCB中的状态由执行改为阻塞，并将PCB插入阻塞队列

当一个进程因为某个事件无法继续执行时，它会被加入到相应的阻塞队列里，等待事件完成。此时，操作系统会调用调度程序，从就绪队列中选择另一个可运行的进程，并进行上下文切换：保存当前进程状态，加载新进程状态，让新进程继续执行。

进程唤醒过程（有关进程(比如提供数据的进程)调用唤醒原语**wakeup**)

- 1.把要唤醒的进程从阻塞队列中移出
- 2.将现行状态修改为就绪
- 3.将该PCB插入到就绪队列中

进程的挂起

引起进程挂起的事件：用户进程请求将自己挂起、父进程请求挂起子进程等

进程挂起过程（利用挂起原语**suspend()**将指定进程或处于阻塞状态的进程挂起。）

- 1.首先检查被挂起进程的状态：活动就绪——》静止就绪；活动阻塞——》静止阻塞
- 2.该进程的PCB复制到某指定的内存区域（方便用户或父进程考查该进程的运行情况）
- 3.若被挂起的进程正在执行，则转向调度程序重新调度。（如果这个进程正在执行，还需要切换出它，重新调度其他进程运行）

进程的激活

引起进程激活的事件：父进程或用户进程请求激活指定进程，若该进程驻留在外存而内存中已有足够的空间时，则可将在外存上处于静止就绪状态的进程换入内存

进程激活过程（利用激活原语**active**将指定进程激活）

- 1.将进程从外存调入内存，检查进程现行状态：静止就绪——》活动就绪；静止阻塞——》活动阻塞
- 2.在抢占式调度策略中，每当有新进程进入就绪队列，操作系统就会比较它和当前运行进程的优先级。如果新进程优先级更高，则立即切换执行它；否则保持当前进程继续运行。

进程同步

引入进程概念就可以使系统多道程序并发执行

临界资源一次只能供一个进程使用

临界区：每个进程中访问临界资源的那段代码

进程之间两种形式的制约关系：

间接相互制约关系：进程共享相同资源

直接相互制约关系：进程和进程相互合作

同步机制遵循的规则

空闲让进：当无进程处于临界区内时，必须让一个要求进入临界区的进程立即进入，以有效地利用临界资源

忙则等待：当已有进程处于临界区内时，其它试图进入临界区的进程必须等待，以保证它们互斥地进入临界区

有限等待：对要求进入临界区的进程，应在有限时间内使之进入，以免陷入“死等”

死锁避免：对于等待进入临界区的进程而言，应允许它无限期等待，以免进程“饥饿”

```
boolean TS(boolean *lock)
{
    boolean old;
    old=*lock;
    *lock=TRUE;
```

临界区：当进程处于临界区时，其它进程进入临界区的进程必须等待，除非它的等待时间超过了规定的等待时间。

有限等待：对要求进入临界区的进程，应在有限时间内使之进入，以免陷入“死等”

让权等待：对于等待进入临界区的进程而言，它必须立即释放处理机，以免进程“忙等”

硬件同步机制（软件方法解决进程互斥有局限性，采用硬件指令来解决）

关中断

锁测试前关闭中断，锁测试完成并上锁打开中断

缺点

- 1.滥用关中断权力可能导致严重后果
- 2.关中断时间过长会影响系统效率，限制处理器交叉执行程序的能力
- 3.不适合多CPU系统，不能防止进程在其他处理器上执行相同的临界段代码

Test-and-Set指令实现互斥

Swap指令实现互斥

信号量机制（硬件同步机制会导致其他访问忙碌临界资源的进程会不断的测试，处于忙等状态，不符合让权等待）

整型信号量：用于表示资源数目的整型量S，但是仍然会存在忙等现象

记录型信号量：需要一个用于代表资源数目的整型变量value外，还应增加一个进程链表指针list，用于链接上述的所有等待进程，它通过阻塞进程而不是让进程空转等待来管理资源竞争

AND信号量：每次可以原子操作多个信号量。只有当一组资源全部满足时，等待的进程才能被唤醒。如果这组资源中的任何一个不可用，进程就会被阻塞，每次可以原子操作多个信号量解决了死锁问题

信号量集：批量分配资源之前，必须测试下限阈值

Swait(S, d, d)。此时在信号量集中只有一个信号量S，但允许它每次申请d个资源，当现有资源数少于d时，不予分配。

Swait(S, 1, 1)。此时的信号量集已蜕化为一般的记录型信号量(S > 1时)或互斥信号量(S = 1时)。

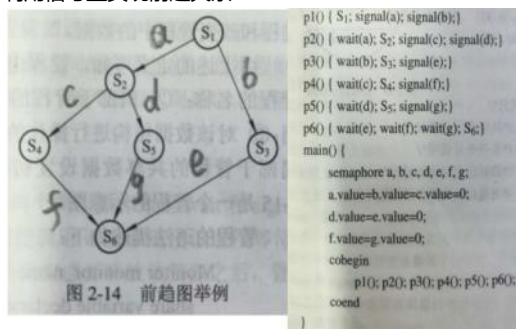
Swait(S, 1, 0)。这是一种很特殊且很有用的信号量操作。当S ≥ 1时，允许多个进程进入某特定区；当S变为0后，将阻止任何进程进入特定区。换言之，它相当于一个可控开关。

信号量的应用：

利用信号量实现进程互斥：

```
Var mutex:semaphore := 1;
begin
  parbegin
    process A: begin
      repeat
        wait(mutex);
        critical section
        signal(mutex);
        remainder section
      until false;
    end
    process B: begin
      repeat
        wait(mutex);
        critical section
        signal(mutex);
        remainder section
      until false;
    end
  end
parend
```

利用信号量实现前趋关系



管程机制

过程封装，进程访问临界资源时，通过管程实现，允许一个进程进入管程

由命名、共享变量说明、操作说明、数据初始化四种语句构成

由此可见，管程相当于围墙，它把共享变量和对它进行操作的若干过程围了起来，所有进程要访问临界资源时，都必须经过管程（相当于通过围墙的门）才能进入，而管程每次只准许一个进程进入管程，从而实现了进程互斥。

条件变量：必须设置两个同步操作原语wait和signal，当某进程通过管程请求临界资源而未能满足时，管程便调用wait原语使该进程等待，并将它排在等待队列上。仅当另一进程访问完并释放之后，管程又调用signal原语，唤醒等待队列中的队首进程。

```
{ boolean old;
  old=*lock;
  *lock=TRUE;
  return old;
}

void swap(boolean *a,boolean *b)
{ boolean temp;
  temp=*a;
  *a=*b;
  *b=temp;
}
```

P/wait(S): while S ≤ 0;

S := S - 1;

V/signal(S): S := S + 1;

```
procedure wait(S)
var S: semaphore;
begin
  S.value := S.value - 1;
  if S.value < 0 then block(S.L)
end
procedure signal(S)
var S: semaphore;
begin
  S.value := S.value + 1;
  if S.value ≤ 0 then wakeup(S.L);
end
```

管程的语法如下：

```
type monitor-name=monitor
variable declarations
procedure entry P1(...);
begin ... end;
procedure entry P2(...);
begin ... end;
...
procedure entry Pn(...);
begin ... end;
begin
  initialization code;
end
```

进程通信

进程的同步与互斥也存在进程通信，因为效率甚低，成为低级通信方式

高级通信方式：进程间可直接利用操作系统所提供的一组通信命令（或原语）来传送大量数据的通信方式

进程通信是指相关进程之间所进行的信息交换

实现进程通信的方式

共享存储器方式：进程通过共享某些数据结构或存储区来进行通信，可分为共享数据结构方式、共享存储区方式

消息传递通信方式：进程间信息交换以消息或报文为单位，利用一组原语来实现通信，分为直接、间接通信方式

共享文件方式：利用共享文件来实现进程间的通信。在unix系统中，利用一个打开的共享文件来连接两个相互通信的进程，该共享文件称为管道，该方式称为管道通信。为了协调双方通信，管道通信必须提供三方面的协调能力：互斥、同步、对方是否存在

客户机-服务器方式：套接字（网通通信接口）、远程过程调用（一个通信协议，用于通过网络连接的系统）

线程的基本概念

为了减少程序在并发执行时所付出的时空开销，使OS具有更好的并发性

线程与进程的比较：

1. 调度的基本单位

进程每次切换上下文调度进程时开销较大

线程切换时只需保存和设置少量寄存器内容，切换代价远低于进程

同一进程中，线程的切换不会引起进程的切换，但是跨进程切换线程则会引起进程的切换

2. 并发性

进程可以并发执行，相同或不同的进程的线程之间也可以并发执行，提高了系统资源的利用率和系统的吞吐率

3. 拥有资源

进程可以拥有资源并作为拥有资源的基本单位，线程本身并不拥有系统资源，而且仅有必不可少的、能保证独立运行的资源（比如TCB、程序计数器、特殊的寄存器和堆栈等）

线程共享进程所拥有的资源：同一进程的所有线程都具有相同的地址空间（线程可以访问该地址空间的每一个虚地址），线程可以访问进程所拥有的资源

4. 独立性

同一进程的线程的独立性远低于不同进程之间的独立性：线程往往是为了提高并发性以及进行相互作用之间的合作而创建的，他们共享进程的内存地址空间和资源，允许其他线程读写

5. 系统开销

线程远小于进程

6. 支持多处理机系统

单线程进程只会在一个处理机上运行，多线程进程可以分配线程到多个处理及上

线程运行的三个状态

执行状态：表示线程已获得处理机而正在运行

就绪状态：指线程已具备了各种执行条件，只须再获得CPU便可立即执行

阻塞状态：指线程在执行中因某事件受阻而处于暂停状态，例如，当一个线程执行从键盘读入数据的系统调用时，该线程就被阻塞

线程控制块TCB

用于记录所有用于控制和管理线程的信息（线程标识符、一组寄存器、线程运行状态、优先级等）

线程的实现方式

内核支持线程KST

内核根据该TCB而感知某线程的存在，并对其加以控制。'

优点

在多处理器系统中，内核能够同时调度同一进程中的多个线程并行执行；

如果进程中的一个线程被阻塞了，内核可以调度该进程中的其它线程占有处理器运行，也可以运行其它进程中的线程；

内核支持线程具有很小的数据结构和堆栈，线程的切换比较快，切换开销小；

内核本身也可以采用多线程技术，可以提高系统的执行速度和效率。

缺点

创建、销毁、上下文切换的开销相对较大

用户级线程ULT

用户空间中实现，与内核无关

优点

线程切换不需要转换到内核空间。

调度算法可以是进程专用的。

用户级线程的实现与OS平台无关，因为对于线程管理的代码是属于用户程序的一部分，所有的应用程序都可以对之进行共享。

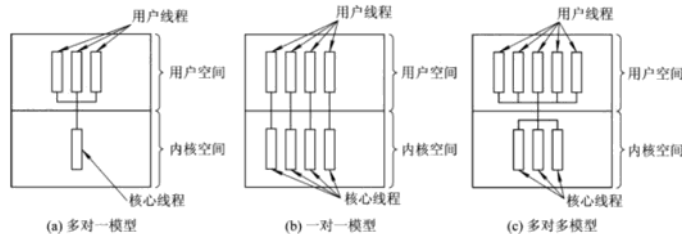
缺点：

若一个线程阻塞（如执行系统调用），整个进程都可能阻塞

无法利用多核 CPU 并行执行（在单核上协作切换）

组合方式ULT/KST 线程

在组合方式线程系统中，内核支持多个内核支持线程的建立、调度和管理，同时，也允许用户应用程序建立、调度和管理用户级线程



多线程模型

线程的实现

内核支持线程的实现

一种可能的线程控制方法是，系统在创建一个新进程时，便为它分配一个任务数据区PTDA（包括若干个TCB空间）

用户级线程的实现（用户级进程都运行在中间系统上，实现中间系统有两大方式：运行时系统、内核控制线程）

运行时系统

用于管理和控制线程的函数(过程)的集合，其中包括用于创建和撤消线程的函数、线程同步和通信的函数，以及实现线程调度的函数等（有这些函数，才能使用户级线程与内核无关，所有函数都驻留在用户空间，并作为用户级线程与内核之间的接口）

内核控制线程

也称轻型进程LWP，每一个进程都可拥有多个LWP，同用户级线程一样，每个LWP都有自己的数据结构(如TCB)，LWP可通过系统调用来获得内核提供的服务，用户及线程运行时，将它连接到一个LWP上便具有了内核支持线程的所有属性

线程的创建和实现

线程的创建

应用程序在启动时，通常仅有一个线程在执行，人们把线程称为“初始化线程”，它的主要功能是用以创建新线程。

在创建新线程时，需要利用一个线程创建函数(或系统调用)，并提供相应的参数，如指向线程主程序的入口指针、堆栈的大小，以及用于调度的优先级等。

在线程的创建函数执行完后，将返回一个线程标识符供以后使用

线程的终止

一个线程完成了自己的任务(工作)后，或是线程在运行中出现异常情况而须被强行终止时，由终止线程通过调用相应的函数(或系统调用)对它执行终止操作。

但有些线程(主要是系统线程)，它们一旦被建立起来之后，便一直运行下去而不被终止。

在大多数的OS中，线程被中止后并不立即释放它所占有的资源，只有当进程中的其它线程执行了**分离函数**后，被终止的线程才与资源分离，此时的资源才能被其它线程利用

处理机调度的层次调度算法的目标

调度的实质是一种资源分配，处理机调度是对处理机资源进行分配

处理机调度的层次

高级调度：调度对象是作业，主要应用于批处理系统。

低级调度：对象是进程（或内核级线程），主要用于批处理系统、分时系统和实时系统中

中级调度：目的是提高内存利用率和系统吞吐量。把那些暂时不能运行的进程，调至外存等待

处理机调度算法的共同目标

资源利用率：使系统中的处理机和其它所有资源都尽可能地保持忙碌状态

$$\text{CPU 的利用率} = \frac{\text{CPU 有效工作时间}}{\text{CPU 有效工作时间} + \text{CPU 空闲等待时间}}$$

公平性：公平性是相对的，对相同类型的进程应获得相同的服务；对于不同类型的**进程**根据其紧急程度或重要性的不同提供不同的服务

平衡性：为使系统中的CPU和各种外部设备都能经常处于忙碌状态，调度算法应尽可能保持**系统资源**使用的平衡性

策略强制执行：对所制订的策略，必须予以准确地执行，即使会造成某些工作的延迟也要执行。

批处理系统的目标

平均周转时间短

系统吞吐量高（单位时间内系统所完成的作业数）

处理机利用率高

分时系统的目标

响应时间快

均衡性

实时系统的目标

截止时间的保证

可预测性

作业与作业调度

作业和作业步

作业：包含程序、数据、作业说明书（系统根据说明书对程序的运行进行控制），以作业为基本单位从外存调入内存

作业步：作业运行期间，每个作业经过的若干个相对独立又相互关联的顺序加工步骤

作业控制块（JCB）

作业在系统中存在的标志，保存了系统对作业进行管理和调度所需的全部信息（作业标识、用户名称、用户帐号、作业类型（CPU繁忙型、I/O繁忙型、批量型、终端型）等）

作业运行的三个阶段和三个状态

收容阶段-后备状态

运行阶段-运行状态

完成阶段-完成状态

作业调度的主要任务

根据JCB中的信息，检查系统中的资源能否满足作业对资源的需求

按照一定的调度算法，从外存的后备队列中选取某些作业调入内存，并为它们创建进程、分配必要的资源。

将新创建的进程排在就绪队列上等待调度

每次执行作业调度时，都需做出以下两个决定。

1. 接纳多少个作业

2. 接纳哪些作业

调度算法

先来先服务(FCFS)

短作业优先(SJF)

不利于长作业，未考虑紧迫性，作业长短用户估计、并不能做到真正的短作业优先

优先级调度算法

高响应比优先调度算法

$$\text{优先权} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}}$$

进程调度

进程调度的任务

保存处理机的现场信息

按某种算法选取进程

把处理器分配给进程

进程调度机制的三个基本部分

排队器（根据策略将就绪进程排成一个或多个队列）

分派器（使用调度算法指派进程）

上下文切换器（包括切出和切入）

进程调度方式

非抢占方式：处理机分配给进程后直到进程完成或被阻塞才会把处理机分配给其他进程

抢占方式：调度程序根据某种原则，暂停正在执行的进程，将已分配给该进程的处理机重新分配给另一进程，比较复杂，开销较大

对于批处理系统，可以防止长进程占用处理机，从而提供更公平的服务

对于分时系统，只有这样才能实现人机交互

对于实时系统，可以满足实施任务的需求

轮转调度算法

根据FCFS原则排序就绪队列，每个时间片按顺序应用于队列上的进程，若时间片内执行完则立即进行下一个进程，若未能完成，则将进程排到末位，执行下一个进程，时间片的选取应该是略大于一次典型的交互所需的时间

优先级调度算法

静态优先级（创建进程时确定的，依据进程类型、进程对资源的需求、用户要求）

动态优先级（创建之初先赋予一个优先级、随进程推进和等待时间的增加而改变）

多队列调度算法

设置多个队列，针对不同的用户需求，更加适配于多处理机系统

多级反馈队列调度算法

如果进程在一个时间片内完成，直接结束运行

如果未完成，就被降级到下一优先级的队列

从第1到第(n-1)队列一般采用FCFS + 时间片

第n队列采用纯轮转（Round Robin），不再继续降级

只有当前优先级队列为空，才会调度下一等级队列中的进程

如果高优先级队列有新进程到来，会立即抢占CPU

当前低优先级进程被挂起并放回其队列末尾，等待下一次调度。



第一个队列的时间片应该略大于多数人机交互所需的处理时间

基于公平原则的调度算法（我赌他不考）

实时调度

实现实时调度的基本条件

必要信息：就绪时间、开始截止时间和完成截止时间、处理时间、资源要求、优先级

系统处理能力强

若采用单处理机系统，须增强其处理能力，以显著地减少对每一个任务的处理时间；

采用多处理机系统。

采用抢占式调度机制

优先权更高的任务到达时，允许将当前任务暂时挂起，而令高优先权任务立即投入运行

具有快速切换机制

对外部中断的快速响应能力。

快速的任务分派能力

实时调度算法（我赌他不考）

死锁概述

资源问题

可重用性资源：一次只能分配给一个进程，请求、使用、释放流程、进程无法创建这样的资源

消耗性资源：资源数目动态变化，进程可以创建资源，可以消耗资源且不归还

可抢占型资源

不可抢占型资源

引起死锁的原因

竞争不可抢占型资源

竞争可消耗资源

进程推进顺序不当（进程对资源申请和释放的顺序不合理）

死锁定义

一组进程中的每一个进程都在等待仅有该组进程中的其他进程才能引发的事件，这组进程就是死锁的

产生死锁的必要条件

互斥条件：一段时间内某资源仅为一个进程所占用。

请求和保持条件：当进程因请求资源而阻塞时，对已获得的资源保持不放。

不可抢占条件：进程已获得的资源在未使用完之前，不能剥夺，只能在使用完时由自己释放。

环路等待条件：在发生死锁时，必然存在一个进程--资源的环形链。

预防死锁

预防死锁的方法是通过破坏产生死锁的四个必要条件中的一个或几个，以避免发生死锁

破坏“请求和保持”条件

第一种协议

所有进程在开始运行之前，必须一次性地申请其在整个运行过程中所需的全部资源。

资源被严重浪费，经常发生饥饿现象

第二种协议

它允许一个进程只获得运行初期所需的资源后，便开始运行。

破坏“不可抢占”条件

一个已经保持了某些不可被抢占资源的进程，提出新的资源请求而不能得到满足时，它必须释放已经保持的所有资源，待以后需要时再重新申请

破坏“循环等待”条件

对系统所有资源类型进行线性排序，并赋予不同的序号

避免死锁

在资源动态分配过程中，防止系统进入不安全状态，以避免发生死锁

安全状态，是指系统能按某种进程顺序($P_1, P_2, \dots, P_n$)(称 $\langle P_1, P_2, \dots, P_n \rangle$ 序列为安全序列)，来为每个进程 P_i 分配其所需资源，直至满足每个进程对资源的最大需求，使每个进程都可顺利地完成。如果系统无法找到这样一个安全序列，则称系统处于不安全状态
银行家算法题自己复习吧

死锁的检测与解除

检测：通过对资源分配图边的逐个消去、简化到消去所有的边，则说明没有死锁，反之则发生死锁

解除：从一个或多个进程中抢占足够的资源分配给死锁进程；终止所有死锁进程或逐个终止进程直到死锁解除

第四章 存储器管理

2025年5月5日 18:46

存储器的层次结构

对于通用计算机，存储层至少三层：cpu寄存器、主存（高速缓存、主存储器、磁盘缓存）、辅存(固定磁盘、可移动存储介质)

可执行存储器：指寄存器和主存储器，采用的访问机制和耗费的时间不同与辅存不同，访问辅存信息需要通过I/O设备，访问可执行存储器可在很短时钟周期内使用load和store

主存储器：内存，保存进程运行时的程序和数据。为缓和主存储器的访问速度远低于CPU执行指令的速度，在计算机系统中引入了寄存器和高速缓存。

寄存器：与CPU协调工作，用于加速存储器的访问速度

高速缓存：根据程序执行的局部性原理将主存中一些经常访问的信息存放在高速缓存中，减少访问主存储器的次数，大幅提高执行速度

磁盘缓存：将频繁使用的一部分磁盘数据和信息，暂时存放在磁盘缓存中，可减少访问磁盘的次数

程序的装入和链接

用程序变为可在内存中执行的程序的通常步骤

编译：由编译程序将用户源代码编译成若干个目标模块。

链接：由链接程序将编译后形成的目标模块以及它们所需要的库函数链接在一起，形成一个装入模块

装入：由装入程序将装入模块装入内存。

装入内存有三种方式

绝对装入方式：用户程序并边以后，产生绝对地址的目标代码（绝对地址可在编译或汇编时给出，也可由程序员直接赋予），适用于计算机系统很小且仅能运行单道程序，可以明确知道程序的驻留位置

可重定位方式：根据内存当前使用情况，将装入模块装入到内存的某个位置，装入程序时，对数据地址和指令地址进行重定位（称作静态重定位）

动态运行时装入方式：在把装入模块装入内存后，并不立即把装入模块中的相对地址转换为绝对地址，而是把这种地址转换推迟到程序真正要执行时才进行，装入内存后的所有地址都仍是相对地址。

程序连接依据时间上的不同分为三种

静态链接

在程序运行之前，先将各目标模块及它们所需的库函数，链接成一个完整的装配模块，以后不再拆开。

需要解决两个问题：对相对地址进行修改，变换外部调用符号

装入时动态链接

将用户源程序编译后所得到的一组目标模块，在装入内存时，采用边装入边链接的链接方式

在装入一个目标模块时，若发生一个外部模块调用，将引起装入程序去找出相应的外部目标模块，并将它装入内存，还要按照静态链接方式来修改目标模块中的相对地址

优点是便于修改和更新，便于实现对目标模块的共享（将一个目标模块链接到几个应用模块）

运行时动态链接：

对某些目标模块的链接，是在程序执行中需要该模块时，才对它进行的链接。

在执行过程中，当发现一个被调用模块尚未装入内存时，立即由OS去找到该模块并将之装入内存，把它链接到调用者模块上

可加快程序的装入过程，节省大量的内存空间

连续分配存储管理方式

单一连续分配

只能用于单用户、单任务的操作系统中。

把内存分为系统区和用户区两部分

由于中断向量通常驻留在低地址部分，故OS通常是放在内存的低址部分；用户区是指除系统区以外的全部内存空间，提供给用户使用

固定分区匹配

划分分区可选择大小相等与不相等

一旦分好，则每个分区的大小固定不再变化，且分区的个数也不再改变

一个分区只能容纳一道作业

优点：内存分配、回收算法简单，容易实现。

缺点：主存空间利用率不高，容易造成资源浪费

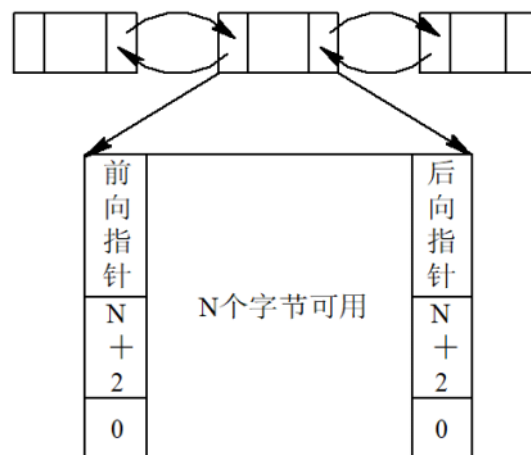
动态分区分配

数据结构

空闲分区表

序号	分区大小 (KB)	分区始址 (KB)	状态
1	64	44	可用
2	24	132	可用
3	40	210	可用
4	30	270	可用
5	...	...	...

空闲分区链



分区分配算法

最佳适应法

寻找其大小最接近于作业所要求的存储区域

会产生很多碎片

最坏适应法

总是寻找最大的空白区

降低产生碎片的可能

首次适应法

总是按地址从低到高搜索，只要找到可以容纳该作业的空白块，就把该空白块分配给该作业

查找开销大，低址部分碎片多，为大作业创造了条件

循环首次适应算法

从上次找到的空闲分区的下一个分区开始找
会缺乏大的空闲分区

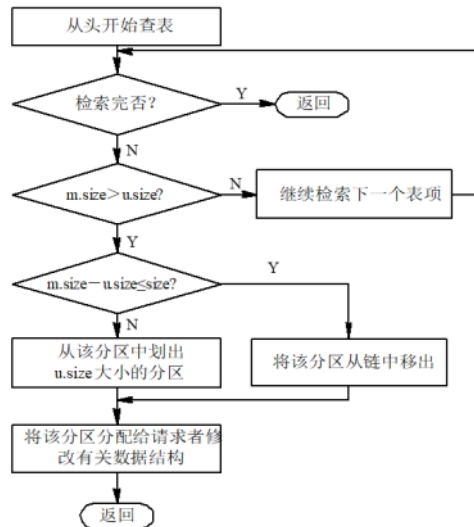
分区分配操作

分配内存

$u.Size$ 是请求分区大小

$m.Size$ 是空闲分区大小

$Size$ 事先规定的不再分割的
剩余分区大小

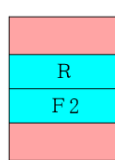


回收内存

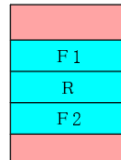
设R：回收区；F1：上邻空白块；F2：下邻空白块



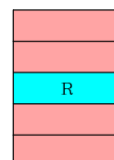
回收区R与上邻
空白块F1邻接



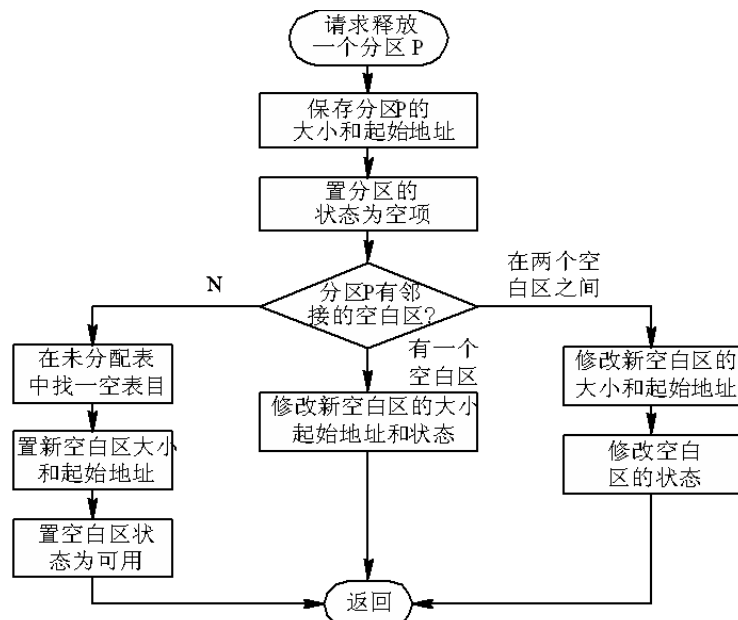
回收区R与下邻
空白块F2邻接



回收区R与上下邻
空白块邻接



回收区R不与
空白块邻接



可重定位分区匹配

紧凑

定时地（或者在主存空间紧张时）把主存中的作业“搬家”集中在主存的一端。另一端就产生了一个大的空闲区

解决了外零头问题，提高了内存利用率

cpu花费时间大，对硬件成本有要求

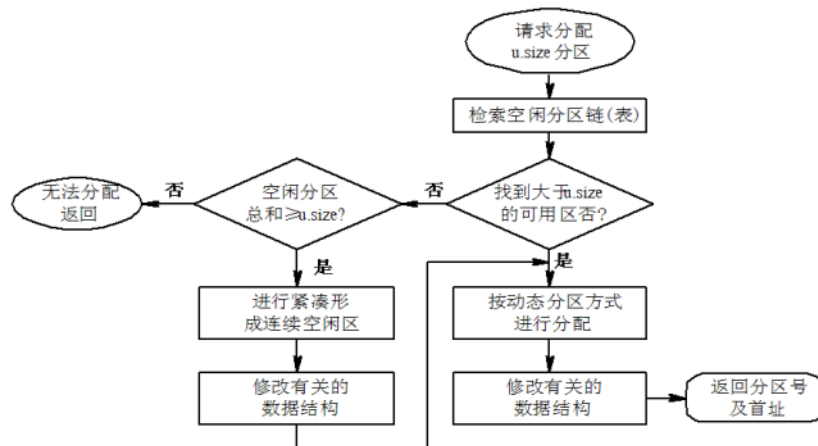
动态重定位

系统中增加一个重定位寄存器，用它来装入程序在内存中的起始地址。

程序在执行时，真正访问的内存地址是相对地址与重定位寄存器中的地址相加而形成。

地址变换过程是在程序执行期间，随着对每条指令和数据的访问而自动进行的，故称为动态重定位。

动态重定位分区分配算法



对换

多道程序环境下的对换技术

对换：把内存中暂时不能运行的进程或者暂时不用的程序和数据，调出到外存上，以便腾出足够的内存空间，再把已具备运行条件的进程或进程所需要的程序和数据，调入内存。提高内存利用率

对换类型：整体对换/进程对换；页面对换；分段对换

系统需要具备三方面功能：管理交换空间、进程换出与换入

对换空间的管理

具有对换功能的OS中通常把外存分为文件区和对换区

文件区管理的主要目标

提高文件存储空间的利用率，其次是提高对文件的访问速度（访问的频率较低），采取离散分配方式的管理方式

对换区管理的主要目标

提高进程的换入、换出速度（驻留的时间是短暂的，而对换操作又较频繁），采取连续分配方式的管理方式

对换区空闲盘块管理中的数据结构

同样可以用空闲分区表或空闲分区链。

空闲分区表中的每个表目中应包含两项，对换区的盘块号和盘块数，

对换空间的分配与回收

与动态分区分配和回收相同。其分配算法可以是首次适应算法、循环首次适应算法或最佳适应算法等

进程的换出与换入

选择被换出的进程

每当一进程由于创建子进程而需要更多的内存空间，但又无足够的内存空间等情况发生时，系统应将某进程换出。

进程换出过程

只能换出不包含共享段的程序和数据

系统先为进程申请对换区中的一块连续空间

系统选择一个优先级较低且处于阻塞状态的进程作为换出对象，启动磁盘 I/O，将该进程的程序段和数据段传送至对换区

传送成功，系统释放该进程所占内存，并在其进程控制块（PCB）中记录对换区位置，同时更新其状态

为“已换出”。

进程的换入

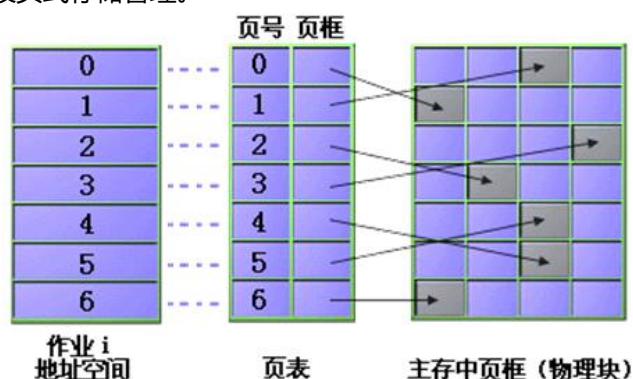
首先查看PCB集合中所有进程的状态，从中找出“就绪”状态但已换出的进程。

当有许多这样的进程时，它将选择其中已换出到磁盘上时间最久的进程作为换入进程，为它申请内存。

如果申请成功，可直接将进程从外存调入内存；如果失败，则需先将内存中的某些进程换出，腾出足够的内存空间后，再将进程调入。

分页存储管理方式

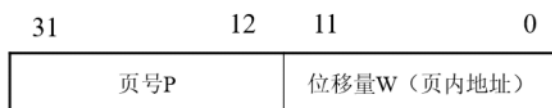
允许将一个进程直接分散地分配到许多不相邻的分区中，就不必再进行“紧凑”。基于这一思想而产生了离散分配方式。根据离散分配时所用基本单位的不同，又可把离散分配方式分为以下三种：分页存储管理、分段存储管理、段页式存储管理。



页面的大小应选择得适中，且页面大小应是2的幂，通常为512B-8KB

地址结构

分页地址中的地址结构如下：

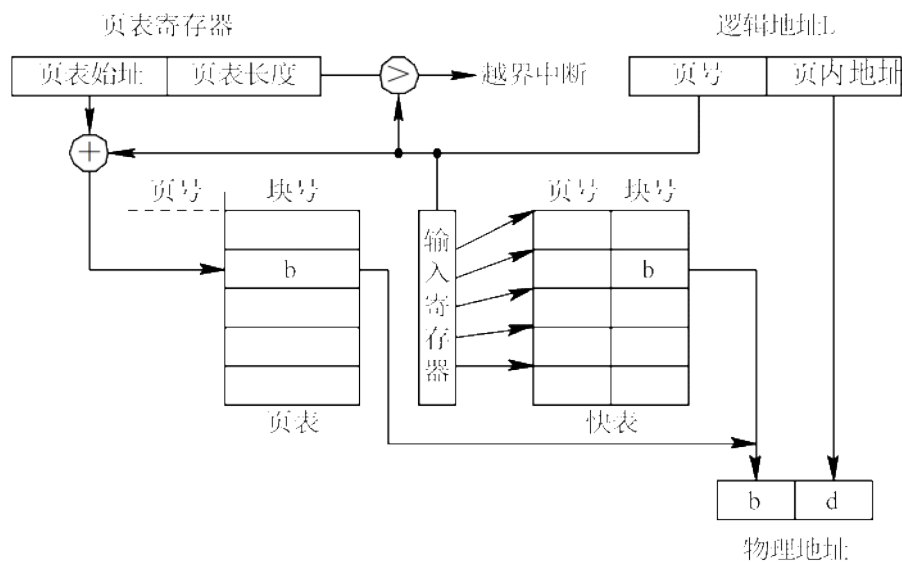


对某特定机器，其地址结构是一定的。若给定一个逻辑地址空间中的地址为A，页面的大小为L，则页号P和页内地址d可按下式求得：

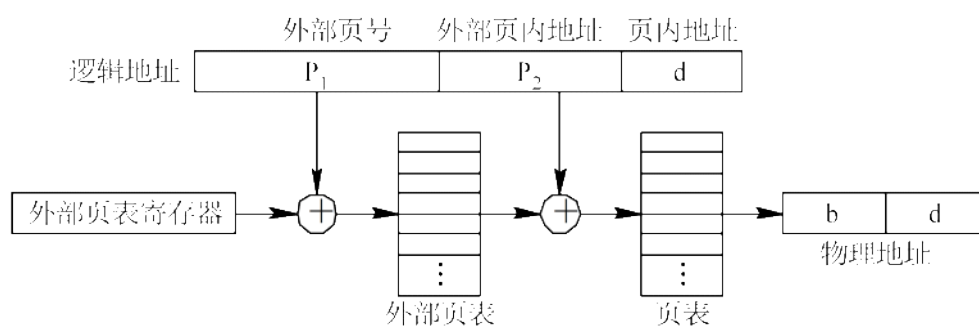
$$P = INT \left[\frac{A}{L} \right]$$

$$d = [A] \bmod L$$

页表列出了作业的逻辑地址与其在主存中的物理地址间的对应关系，通常还包括一个存取控制字段，这个表目也称为页描述子

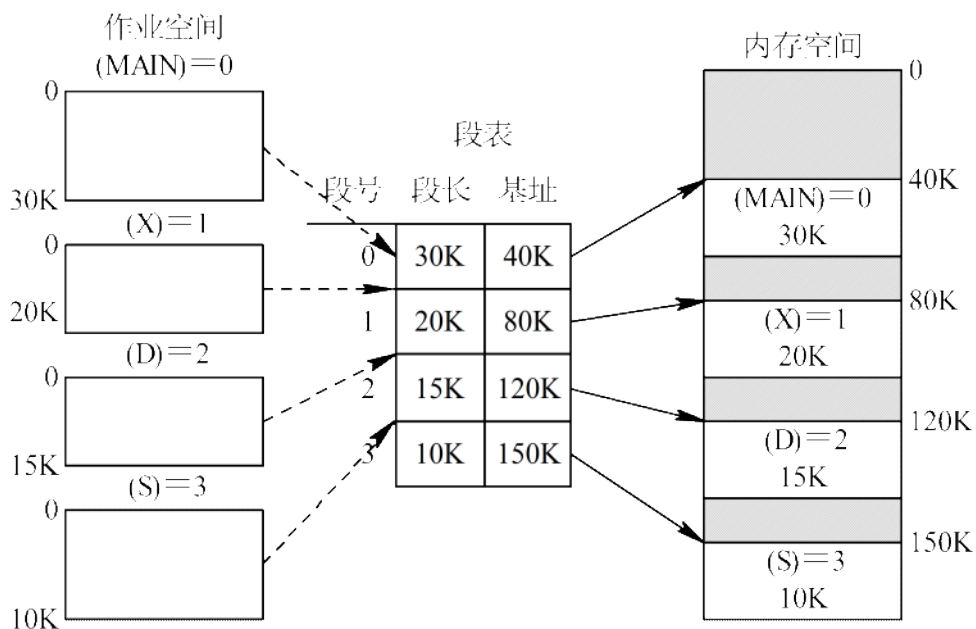


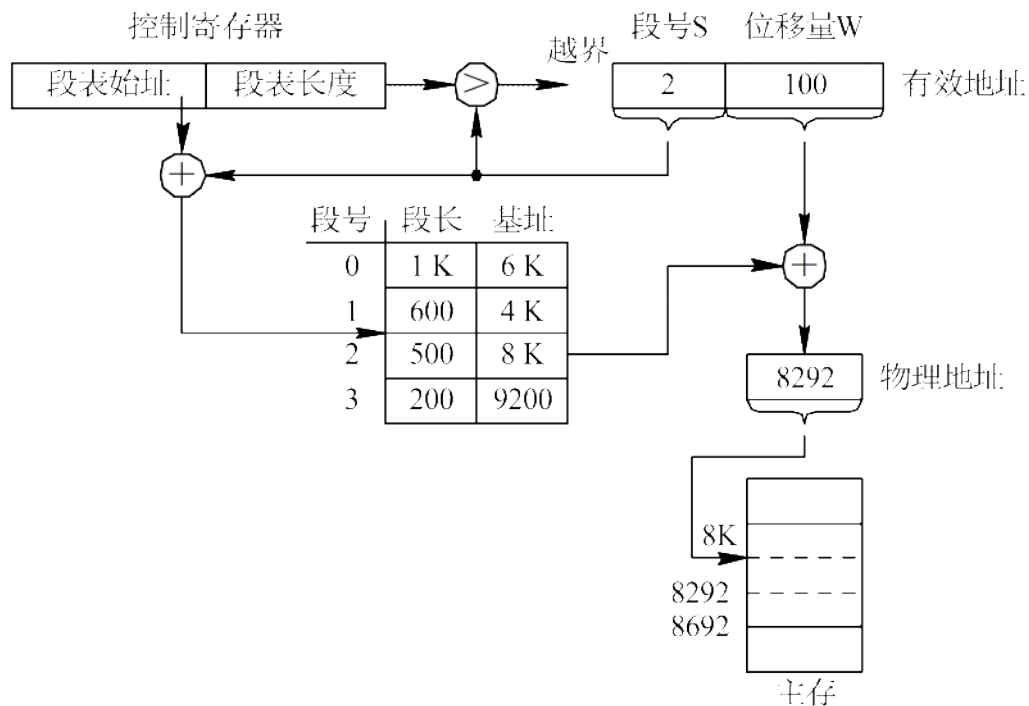
两级和多级页表



分段存储管理方式

主要是为了满足用户在编程和使用上多方面的要求：方便编程、分段共享、信息保护、动态增长、动态链接





分页和分段的区别

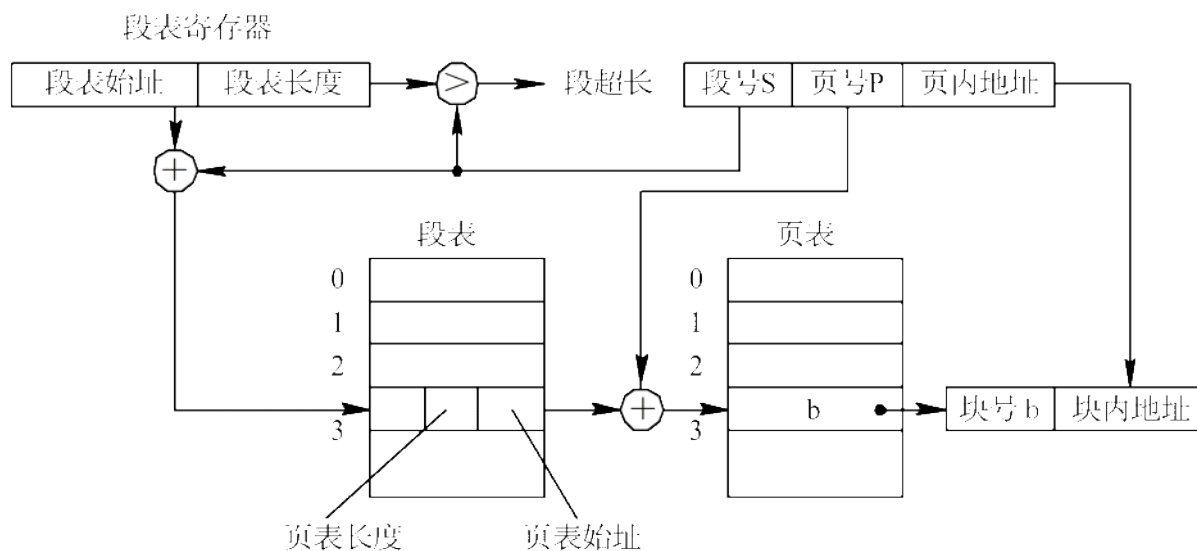
页是信息的物理单位；段则是信息的逻辑单位

分页的作业地址空间是一维的（页号+偏移）；分段的作业地址空间则是二维的（段号和段内偏移）

页的大小固定且由系统决定，系统中只能有一种大小的页面；段的长度却不固定，决定于用户所编写的程序，通常由编译程序在对源程序进行编译时，根据信息的性质来划分

段页式存储管理方式

在段页式系统中，为了获得一条指令或数据，须三次访问内存。



第五章 虚拟存储器

2025年5月6日 12:01

虚拟存储器概述

常规存储器管理方式的特征

一次性：是指在作业运行前，将作业全部装入内存，但实际上有许多作业在每次运行时，并非用到其全部程序

驻留性：作业装入内存后，便一直驻留在内存直至作业运行结束

局部性原理

程序执行时，除了少部分的转移和过程调用指令外，在大多数情况下仍是顺序执行的。

过程调用将会使程序的执行轨迹由一部分区域转至另一部分区域，但经研究表明，过程调用的深度在大多数情况下都不超过5。也就是说，程序将会在一段时间内，都局限在这些过程的范围内运行。

程序中存在许多循环结构，这些虽然只由少数指令构成，但是它们将多次执行。

程序中还包括许多对数据结构的处理，如对数组进行操作，它们往往都局限于很小的范围内

时间局限性：如果程序中的某条指令一旦执行，则不久以后该指令可能再次执行；如果某数据被访问过，则不久以后该数据可能再次被访问。产生时间局限性的典型原因，是由于在程序中存在大量的循环操作

空间局限性：一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也将被访问，即程序在一段时间内所访问的地址，可能集中在一定的范围之内，其典型情况便是程序的顺序执行。

虚拟存储器定义：是指具有请求调入功能和置换功能，能从逻辑上对内存容量加以扩充的一种存储器系统。其逻辑容量由内存容量和外存容量之和所决定，其运行速度接近于内存速度，而每位的成本却又接近于外存。

虚拟存储器的实现方法

请求分页系统

硬件支持

请求分页的页表机制，它是在纯分页的页表机制上增加若干项而形成的，作为请求分页的数据结构；

缺页中断机构，即每当用户程序要访问的页面尚未调入内存时便产生一缺页中断，以请求OS将所缺的页调入内存；

地址变换机构，它同样是在纯分页地址变换机构的基础上发展形成的。

实现请求分页的软件

请求分段系统

硬件支持

请求分段的段表机制，这是在纯分段的段表机制基础上，增加若干项而形成的；

缺段中断机构，每当用户程序要访问的段尚未调入内存时，产生一缺段中断，以请求OS将所缺的段调入内存；

地址变换机构，它同样是在纯分段地址变换机构的基础上发展形成的

实现请求分段的软件

虚拟存储器的特征

离散性：离散性是指在内存分配时采用离散分配的方式，这是其它几个特征的基础。没有离散性，也就不可能实现虚拟存储器。

多次性：多次性是指一个作业被分成多次地调入内存运行，亦即在作业运行时没有必要将其全部装入，只须将当前要运行的那部分程序和数据装入内存即可；以后运行到那一部分时，再将它调入。多次性是虚拟存储器最重要的特征。

对换性：对换性是指允许在作业的运行过程中换进、换出。

虚拟性：是指能够从逻辑上扩充内存容量，使用户所看到的内存容量远大于实际内存容量。

请求分页存储管理方式

系统需要解决下面三个问题：

系统如何获知进程当前所需页面不在主存。

当发现缺页时，如何把所缺页面调入主存。

当主存中没有空闲的页框时，为了要接受一个新页，需要把老的一页淘汰出去，根据什么策略选择欲淘汰的页面。

请求分页中的硬件支持

页表机制

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

状态位P 用于指示该页是否已调入内存，供程序访问时参考。

访问字段A 用于记录本页在一段时间内被访问的次数，或最近已有多长时间未被访问，提供给置换算法选择换出页面时参考。

修改位M 表示该页在调入内存后是否被修改过。作为该页换出时是否写回外存的依据。

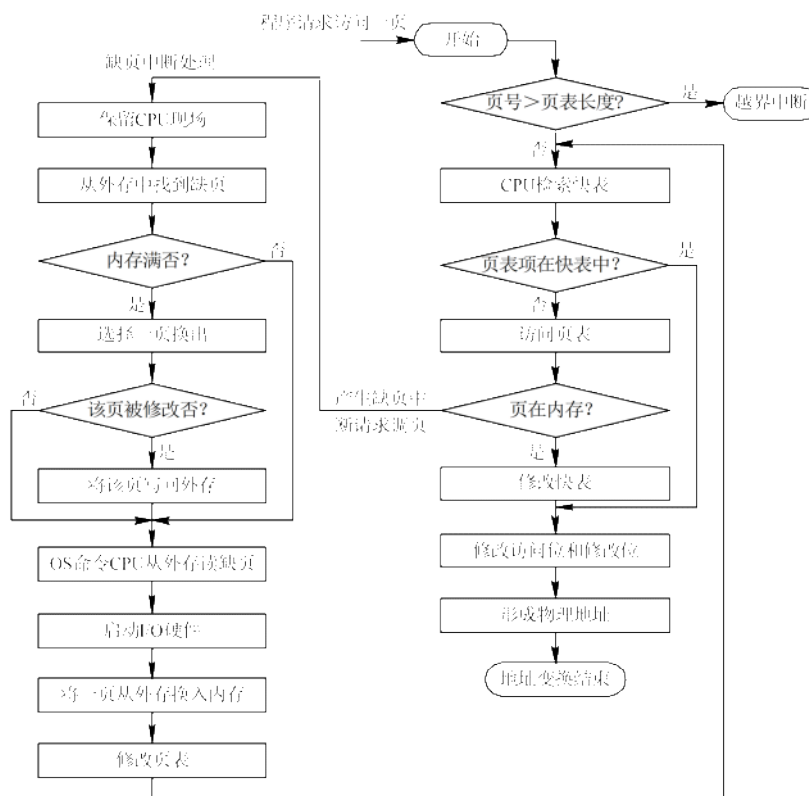
外存地址 用于指出该页在外存上的地址，通常是物理块号，供调入该页时使用。

缺页中断机制

发生在指令执行的过程中产生和处理中断信号（一般中断是在指令执行完才检查是否有中断请求）

系统中的硬件机构应能保存多次中断时的状态，并保证最后能返回到中断前产生缺页中断的指令处

地址变换机构



内存分配策略和分配算法

最小物理块数：指能保证进程正常运行所需的最小物理块数。与计算机的硬件结构有关，取决于指令的格式、功能和寻址方式

物理块的分配策略

固定分配：为每个进程分配一组固定数目的物理块，运行期间不再改变

可变分配：为每个进程分配一组固定数目的物理块，运行期间可做适当的增加或减少

局部置换：如果进程在运行中发现缺页，只能从分配给该进程的n个页面中选出一页换出，然后再调入一页，以保证分配给该进程的内存空间不变

全局置换：如果进程在运行中发现缺页，则将OS所保留的空闲物理块取出一块分配给该进程，或者以所有进程的全部物理块为标的，选择一块换出，然后将所缺之页调入，这样，分配给该进程的内存空间就随之增加

可组合为三种：固定分配局部置换，可变分配全局置换，可变分配局部置换

物理块分配算法

平均分配算法：没办法考虑进程大小

按比例分配算法：根据进程大小分配物理块

考虑优先权的分配算法：通常情况下，一部分按比例地分配给各进程；另一部分则根据各进程的优先权，适当地增加其相应份额后，分配给各进程。有的系统（实时控制系统）也可能完全按照优先权来分配

调页策略

何时调入页面

预调页策略

就是将那些预计在不久之后便会被访问的页面预先调入内存。

请求调页策略

当进程在运行中需要访问某部分程序和数据时，若发现其所在的页面不在内存，便立即提出请求，由OS将其所需页面调入内存。

从何处调入页面

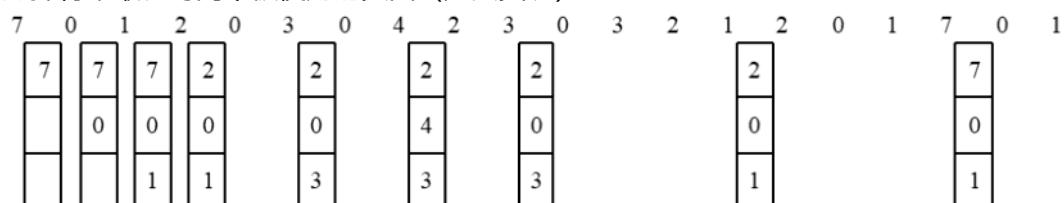
系统拥有足够的对换区空间，这时可以全部从对换区调入所需页面，以提高调页速度

系统缺少足够的对换区空间，这时凡是不会被修改的文件，都直接从文件区调入；而当换出这些页面时，由于它们未被修改而不必再将它们换出，以后再调入时，仍从文件区直接调入。但对于那些可能被修改的部分，在将它们换出时，便须调到对换区，以后需要时，再从对换区调入

页面置换算法

最佳置换算法

选择将来最长时间不被使用的页面（无法实现）

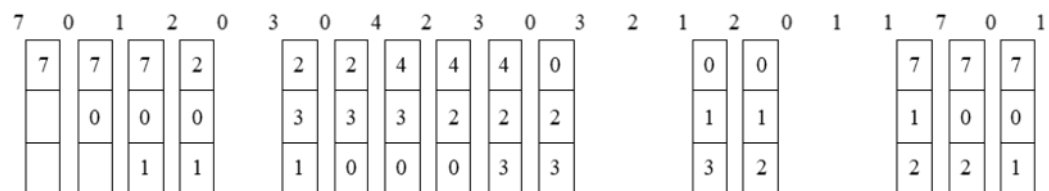


页框（物理块）

利用最佳页面置换算法时的置换图

先进先出置换算法

选择存在时间最久的页面

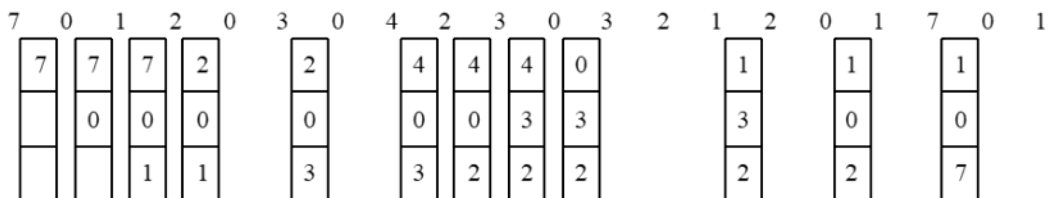


页框

利用FIFO置换算法时的置换图

最近最久未使用(LRU)置换算法

选择最长时间未被访问的页面进行替换（利用局部性原理）



页框

LRU页面置换算法

“抖动” 与工作集

请求分页系统的优缺点：实现了虚拟内存，用小存储运行大程序；会产生缺页中断，若次数变大对换频繁，会导致性能变差

产生抖动的根本原因：进程的增多导致缺页现象的增多，大量的页面换出与换入等待导致进程无法做其它有效工作，只能等待

预防抖动的方法

局部置换策略

在处理机调度中采用工作集算法

操作系统跟踪每个进程的工作集，并为进程分配大于其工作集的物理块。

如果所有工作集之和增加到超过了可用物理块的总数，操作系统会暂停一个进程，将其页面调出并且将其物理块分配给其他进程，防止出现抖动现象。

将阻塞等状态进程调出

对处于等待 I/O、信号、用户输入等阻塞状态的进程，暂时将其页面调出，释放内存。

等待事件完成后再重新调入，有效降低瞬时内存压力

检测并调节

检测缺页率较高的进程，从备用进程块种调入供其使用

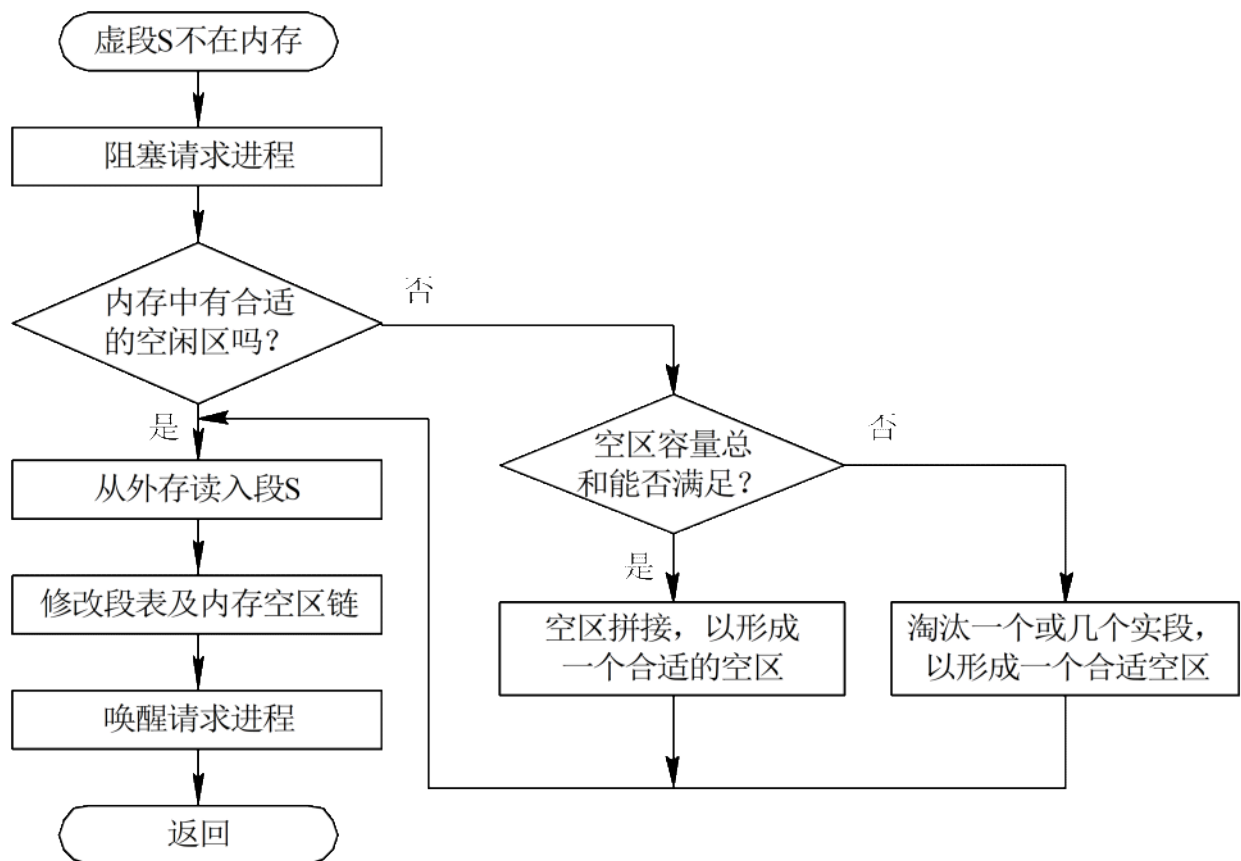
请求分段存储管理方式

段表机制

段名	段长	段的基址	存取方式	访问字段A	修改位M	存在位P	增补位	外存始址

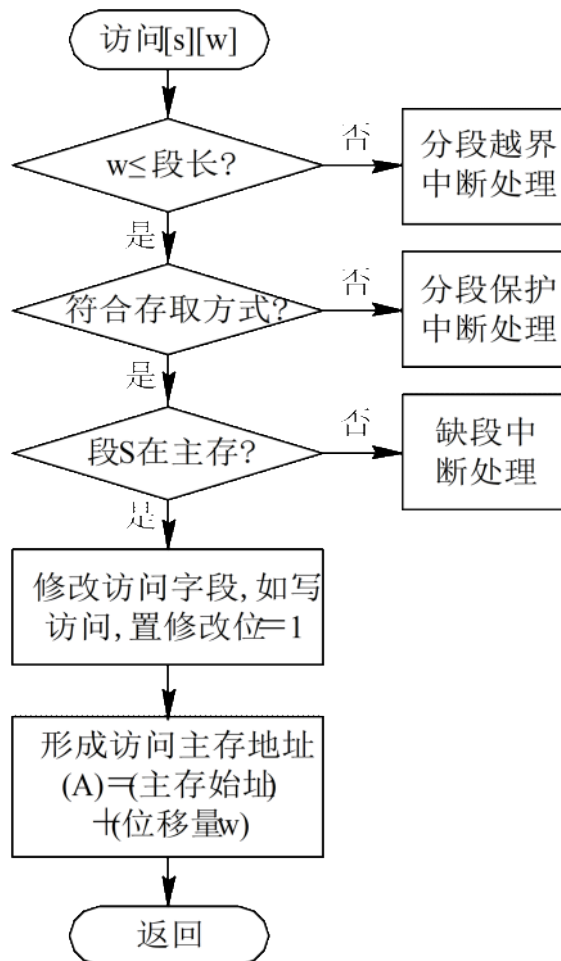
缺段中断机制

请求分段系统中的中断处理过程

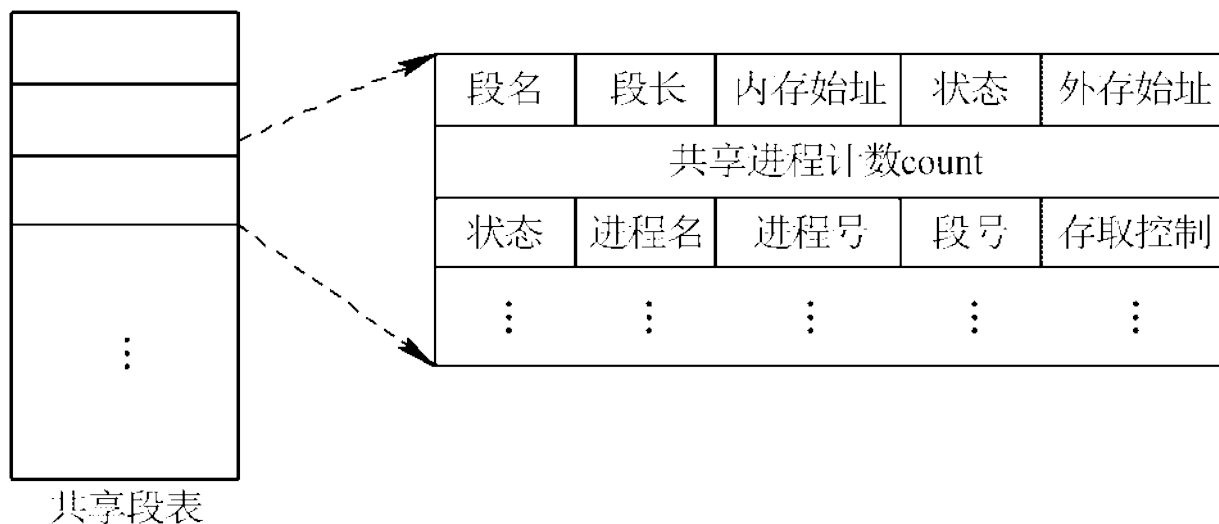


地址变换机构

请求分段系统的地址变换过程



分段的共享与保护



共享段的分配与回收

分配：只有第一次使用时分配物理内存，其后再有进程使用，只需更新它们的段表即可，count每次都修改

回收：只要还有进程使用，就保留；当最后一个进程释放时，才真正释放物理内存并清理系统记录。

分段保护

越界检查

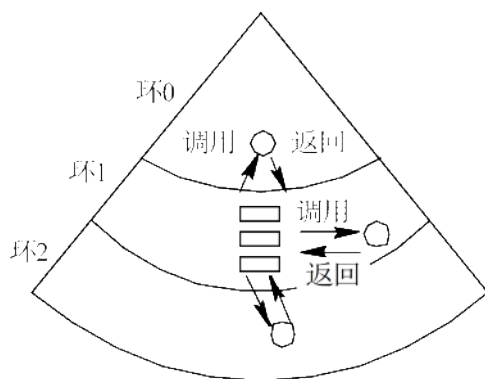
存取控制检查

以段为基本单位检查读写权限

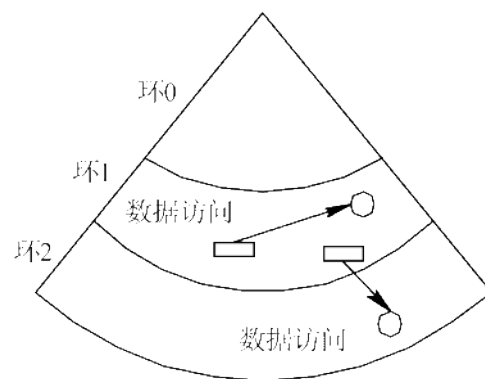
环保护机构

一个程序可以访问驻留在相同环或较低特权环中的数据。

一个程序可以调用驻留在相同环或较高特权环中的服务。



(a) 程序间的控制传输



(b) 数据访问

第六章 输入输出系统

2025年5月8日 0:15

I/O系统的功能、模型和接口

I/O系统管理对象：I/O设备和相应的设备控制器

I/O系统任务：完成用户提出的I/O请求，提高I/O速率

I/O系统目标：提高设备的利用率，为高层进程使用设备提供方法

I/O的基本功能

隐藏物理设备的细节：隐藏底层物理设备的细节，屏蔽不同I/O设备之间在控制方式、速度、数据格式等方面的差异，简化上层软件或用户的使用难度

与设备的无关性：提供设备无关的I/O接口，使应用程序不需要关心所使用的具体设备类型，从而提高系统的可移植性和灵活性

提高处理机和I/O设备的利用率：提高处理器与I/O设备的利用率。一方面，操作系统应快速响应I/O请求，使设备尽早投入运行；另一方面，通过中断、DMA等机制，减少CPU对I/O过程的干预，提高并行效率

对I/O设备进行控制：

轮询的可编程I/O方式：CPU不断主动查询（轮询）I/O设备的状态，判断设备是否准备好进行数据传输

中断的可编程I/O方式：CPU在发出I/O请求后继续执行其他任务，当设备准备好时，通过中断信号通知CPU进行数据处理

直接存储器访问方式:数据在I/O设备和内存之间直接传输，无需CPU参与整个传输过程，只在开始和结束时由CPU控制

I/O通道方式:设置独立的I/O处理器或通道（类似专用子处理器），用于管理多个I/O设备的操作，进一步解放CPU

确保对设备的正确共享

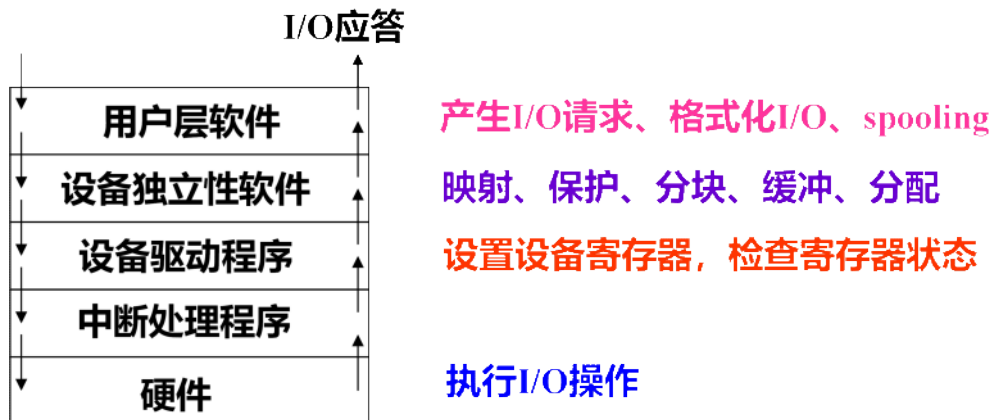
对于独占设备，进程应互斥的访问这类设备，进行分配时，应该考虑到分配的安全性

对于共享设备，允许一段时间内多个进程同时访问设备，进行读写操作时不会影响到正确性
错误处理

能够发现和解决错误和故障，对于临时性错误，重复操作解决，对于持久性错误向上层报告。

I/O系统的层次结构和模型

I/O系统的软件的四个层次



用户层软件：实现与用户交互的接口，用户可直接调用在用户层提供的、与I/O操作有关的库函数，对设备进行操作。

设备独立性软件：负责实现与设备驱动器的统一接口、设备命名、设备的保护以及设备的分配与释放等，同时为设备管理和数据传送提供必要的存储空间，**为设备分配缓存，进行缓存管理**（属于系统级别的逻辑，远高于驱动程序中对硬件寄存器的读写控制）。

设备驱动程序：与硬件直接相关，负责具体实现系统对设备发出的操作指令，驱动I/O设备工作的驱动程序。

中断处理程序：用于保存被中断进程的CPU环境，转入相应的中断处理程序进行处理，处理完后再恢复被中断进程的现场后返回到被中断进程。

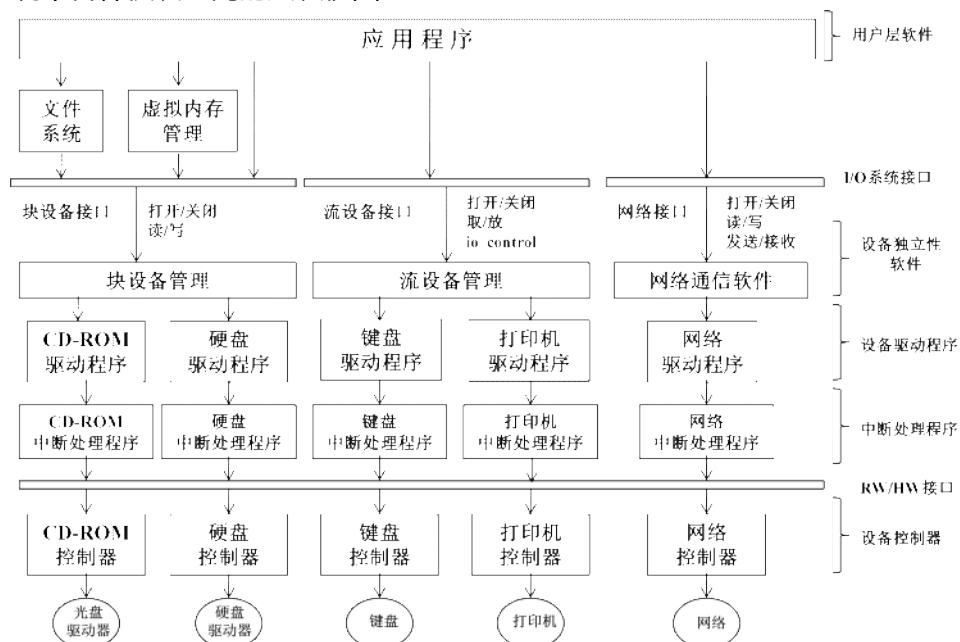
I/O系统中各种模块之间的层次视图

I/O系统的上、下接口

I/O系统接口：是I/O系统与上层系统之间的接口，向上层提供对设备进行操作的抽象I/O命令，以方便高层对设备的使用

软件/硬件(RW/HW)接口：在它的上面是中断处理程序和用于不同设备的设备驱动程序。在它的下面是各种设备的控制器

I/O系统中各种模块之间的层次视图



I/O系统的分层

中断处理程序：它处于I/O系统的底层，直接与硬件进行交互。当有I/O设备发来中断信号时，在中断硬件做了初步处理后，便转向中断处理程序。

设备驱动程序：它处于I/O系统的中间层。是进程与设备控制器之间的通信程序，其作用

是将上层发来的抽象I/O请求转换为对I/O设备的具体命令和参数，并把它装入设备控制器中的命令和参数寄存器中，或相反

设备独立性软件：现代OS中的I/O系统基本上都实现了与设备无关性，也称为与设备无关软件。也就是I/O软件独立于具体使用的物理设备。

I/O系统接口

块设备接口（可寻址，支持随机访问）

块设备：数据的存取和传输都是以数据块为单位的设备（典型磁盘）

隐藏了磁盘地址的二维结构

将抽象命令映射为底层操作（将逻辑块号转化为磁盘的盘面、磁道等）

流设备接口（不可寻址、按顺序读写）

字符设备：数据的存取和传输都是以数据块为单位的设备（键盘、打印机）

get和put操作：从字符缓冲区中取出和输入（字符设备是不可寻址的）

in-control指令：抽象设备功能，让高层程序不必关心具体硬件命令

网络接口

把计算机连接到网络上，操作系统也必须提供相应的网络软件和网络通信接口，使计算机能通过网络与网络上的其它计算机进行通信或上网浏览。

I/O设备和设备控制器

I/O设备的类型

按传输速率分类

低速设备：传输速率仅为每秒钟几个字节至数百个字节的一类设备（键盘、鼠标器、语音）

中速设备：传输速率在每秒钟数千个字节至数万个字节的一类设备（行式打印机、激光打印机）

高速设备：传输速率在数百千个字节至数十兆字节的一类设备（磁带机、磁盘机、光盘机）

按信息交换的单位分类：

块设备：信息的存取总是以数据块为单位（可寻址，支持随机访问）

字符设备：基本单位是字符（不可寻址、按顺序读写）

按设备的共享属性分类：

独占设备

共享设备

虚拟设备

设备与控制器之间的接口

接口中存在三种信号：数据信号、控制信号、状态信号

设备控制器的功能

接收和识别命令--控制寄存器；命令译码

数据交换—CPU(总线)->控制器(数寄)->设备

标识和报告设备的状态—状态寄存器

地址识别—设备和寄存器地址；地址译码器

数据缓冲—用缓冲器暂存由CPU传来的数据

差错控制—差错检测、重发

设备控制器处于CPU与设备之间，它既要与CPU通信，又要与设备通信，还应具有按照CPU所发来的命令去控制设备工作的功能，因此大多设备控制器由以下三部分组成：设备控制器与处理机的接

口、设备控制器与设备的接口、I/O逻辑

内存映射I/O

利用特定的I/O指令：将设备和地址分别设置端口和证书，增加了指令数目、内存地址不会被设备占用，需要指令区分设备和地址操作

内存映射I/O：将设备和地址统一编址，指令集更简单，占用了部分内存地址空间，地址维护比较麻烦

I/O通道(I/O Channel)设备的引入

目的是使一些原来由CPU处理的I/O任务转由通道来承担，从而把CPU从繁杂的I/O任务中解脱出来

I/O通道是一种特殊的处理机

一是其指令类型单一，这是由于通道硬件比较简单，其所能执行的命令，主要局限于与I/O操作有关的指令；再就是通道没有自己的内存，通道所执行的通道程序是放在主机的内存中的，换言之，是通道与CPU共享内存

通道类型

字节多路通道：按字节交叉方式工作的通道，不适于连接高速设备

数组选择通道：一次只选择一个设备，但可成组、成块地传送大量数据，适用高速设备

数组多路通道：结合了数组选择通道与字节多路通道的优点，可同时支持多个低速设备轮流传输字节；同时又能为高速设备批量传输数据

瓶颈问题

由于通道设备数目较低，容易造成数据阻塞，当产生无法走通的路径，将会阻塞数据，需要增加设备到主机间的通路，而不增加通道

中断机构和中断处理程序

中断：中断是指CPU对I/O设备发来的中断信号的一种响应。CPU暂停正在执行的程序，保留CPU环境后，自动地转去执行I/O设备的中断处理程序。执行完后，再返回断点，继续执行原来的程序。由于中断使由外部设备引起的，故又称为中断

陷入：还有一种由CPU内部事件所引起的中断，例如进程在运算中发生了上溢或下溢，又如程序出错，如非法指令、地址越界，以及电源故障等。通常把这类中断称为内中断或陷入

中断向量表：当某个中断源发出中断时，CPU根据中断号查表找到对应服务程序的入口地址，然后跳转执行

中断优先级：根据预设的优先级确定哪个中断先被处理

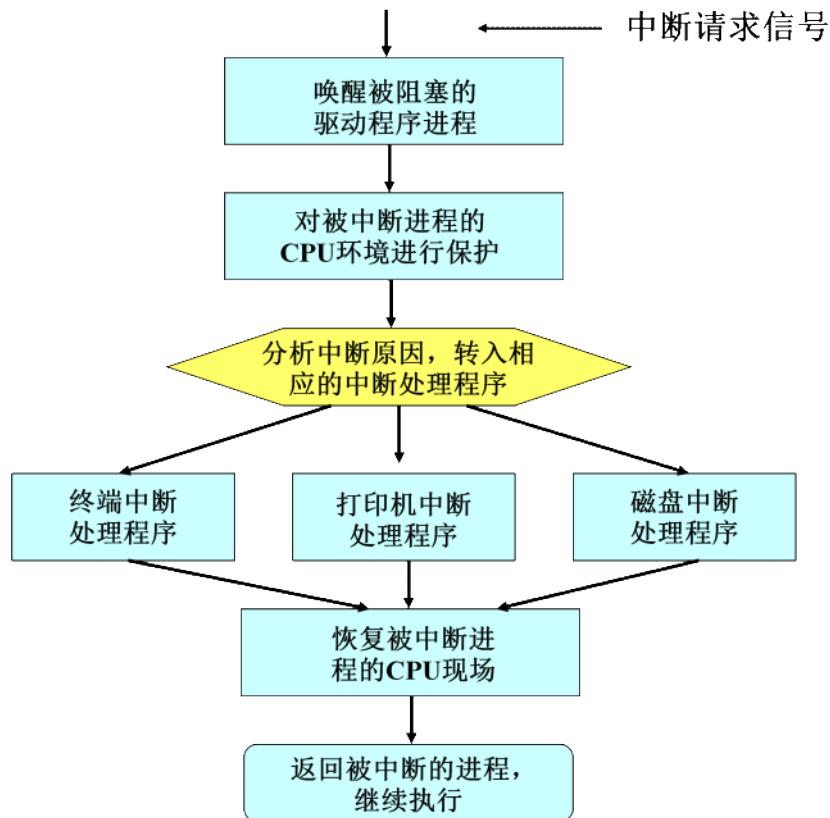
屏蔽（禁止）中断：设置中断屏蔽位，暂时禁止某些中断进入处理流程（保护操作）

嵌套中断：允许优先级更高的中断打断当前正在处理的中断服务程序

中断处理程序

中断处理层的主要工作：进行进程上下文的切换，对处理中断信号源进行测试，读取设备状态和修改进程状态

中断处理流程：



设备驱动程序

是I/O系统的高层与设备控制器之间的通信程序，其主要任务是接收上层软件发来的抽象I/O要求，如read或write命令，再把它转换为具体要求后，发送给设备控制器，启动设备去执行；反之，它也将由设备控制器发来的信号传送给上层软件

驱动程序与硬件密切相关，故通常应为每一类设备配置一种驱动程序

设备驱动程序的功能

接收由I/O进程发来的命令和参数，并将命令中的抽象要求转换为具体要求，例如，将磁盘块号转换为磁盘的盘面、磁道号及扇区号。

检查用户I/O请求的合法性，了解I/O设备的状态，传递有关参数，设置设备的工作方式。

发出I/O命令，如果设备空闲，便立即启动I/O设备去完成指定的I/O操作；如果设备处于忙碌状态，则将请求者的请求块挂在设备队列上等待

及时响应由控制器或通道发来的中断请求，并根据其中断类型调用相应的中断处理程序进行处理。

对于设置有通道的计算机系统，驱动程序还应能够根据用户的I/O请求，自动地构成通道程序。

设备驱动程序的特点

驱动程序主要是指在请求I/O的进程与设备控制器之间的一个通信和转换程序。

驱动程序与设备控制器和I/O设备的硬件特性紧密相关，因而对不同类型的设备应配置不同的驱动程序。

驱动程序与I/O设备所采用的I/O控制方式紧密相关。

由于驱动程序与硬件紧密相关，因而其中的一部分必须用汇编语言书写。

驱动程序应允许可重入。一个正在运行的驱动程序常会在一次调用完成前被再次调用。

驱动程序不允许系统调用

设备处理方式

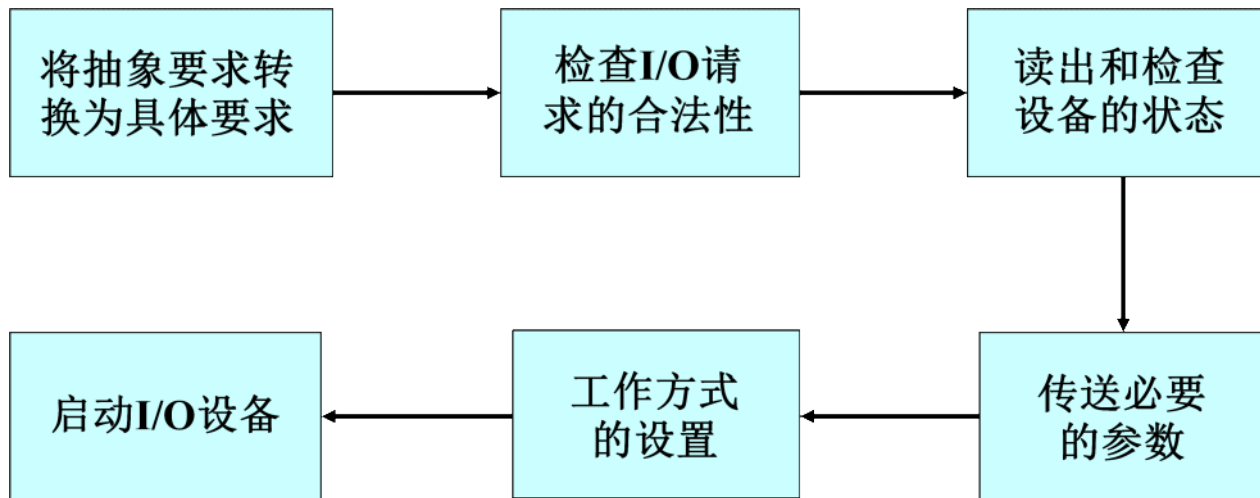
为每一类设备设置一个进程，专门用于执行这类设备的I/O操作，一对一

在整个系统中设置一个I/O进程，专门用于执行系统中所有各类设备的I/O操作。也可以设置一个输

入进程和一个输出进程，一对多

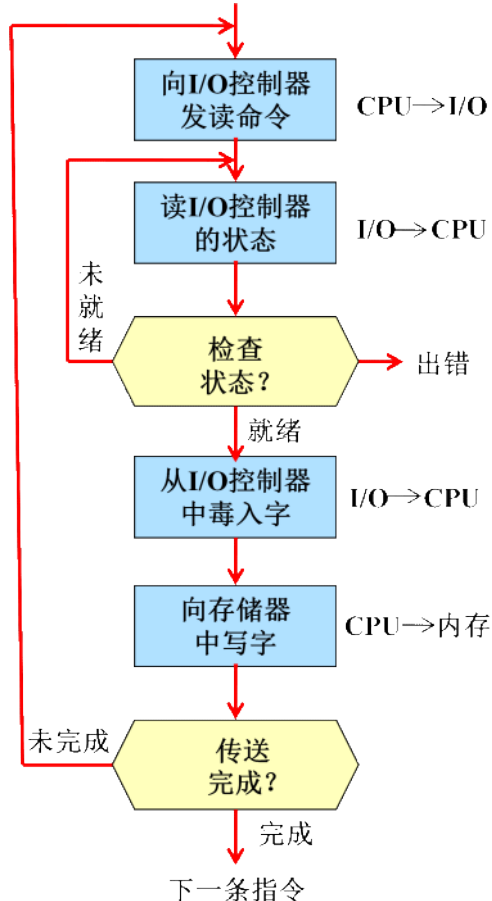
不设置专门的设备处理进程，而只为各类设备设置相应的设备处理程序(模块)，供用户进程或系统进程调用。自助使用已有程序

设备驱动程序的处理过程

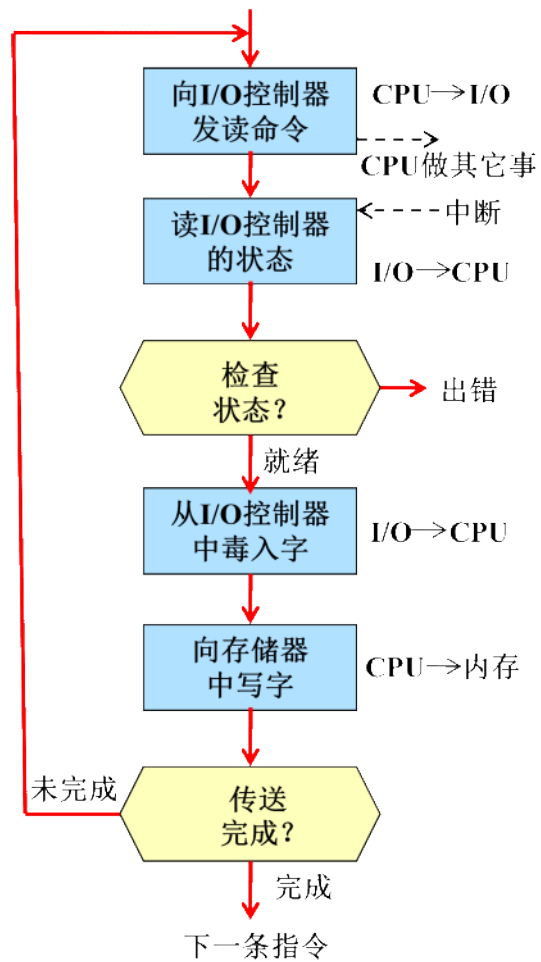


对I/O设备的控制方式

使用轮询的可编程I/O方式（CPU 的绝大部分时间都处于等待I/O设备完成数据I/O的循环测试中，造成对CPU的极大浪费）

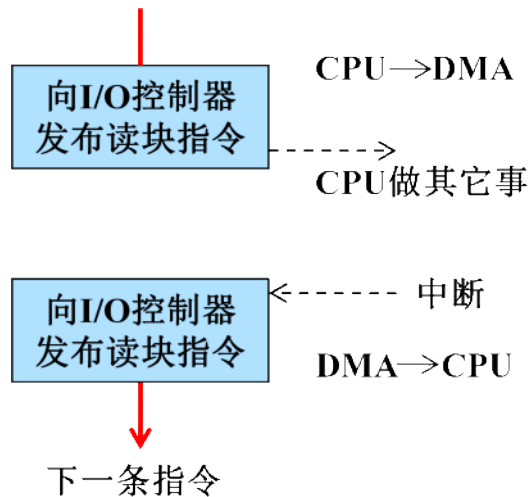


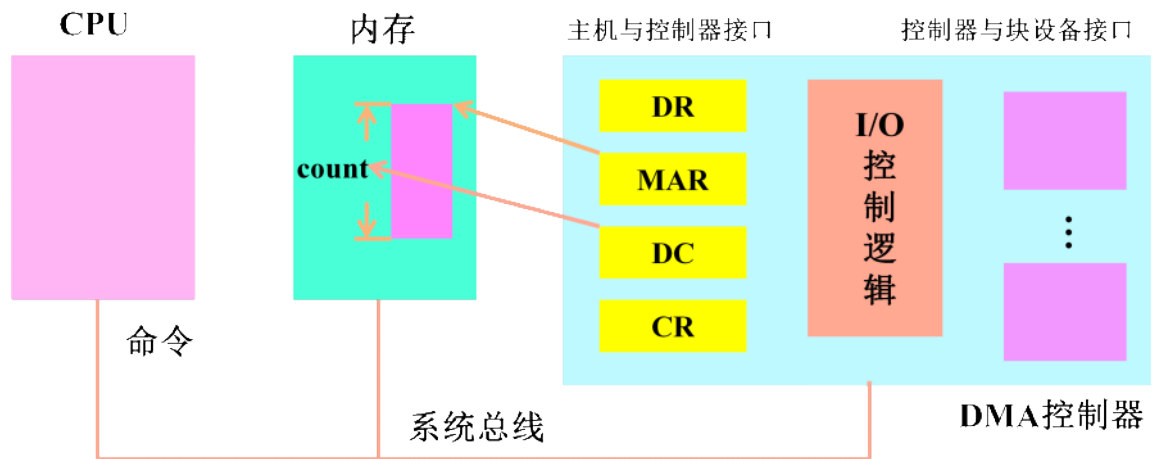
使用中断的可编程I/O方式（仅当输完一个数据时，才需CPU花费极短的时间去做些中断处理）



直接存储器访问方式

数据传输的基本单位是数据块；
 所传输的数据是从设备直接送入内存的，或者相反；
 整块数据的传送是在控制器的控制下完成的；



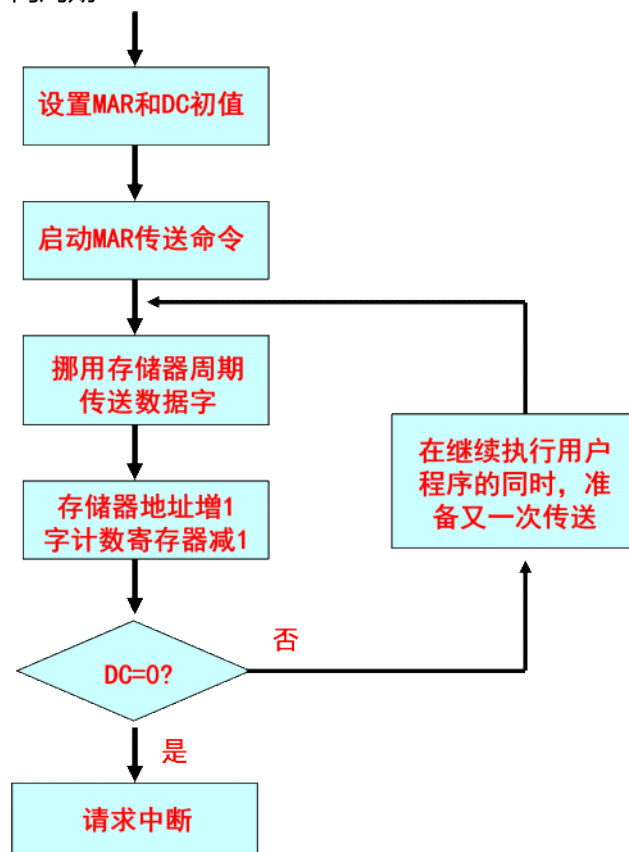


DMA控制器中设置如下四类寄存器

- 命令/状态寄存器CR。用于接收从CPU发来的I/O命令或有关控制信息，或设备的状态。
- 内存地址寄存器MAR。在输入时，它存放把数据从设备传送到内存的起始目标地址；在输出时，它存放由内存到设备的内存源地址。
- 数据寄存器DR。用于暂存从设备到内存，或从内存到设备的数据。
- 数据计数器DC。存放本次CPU要读或写的字(节)数

DMA控制方式

挪用存储器周期传送数据字：在 CPU 某个时钟周期不访问内存时，临时挪用这一个存储器访问周期



I/O通道控制方式

CPU下达启动指令后，通道独立执行通道程序，对控制器发出操作命令，实现设备和内存之间的数据传输

与设备无关的I/O软件

设备的独立性，也称为设备无关性，其基本含义是：应用程序独立于具体使用的物理设备

与设备无关软件的基本概念

在应用程序中，使用逻辑设备名称来请求使用某类设备；而系统在实际执行时，还必须使用物理设备名称。因此，系统须具有将逻辑设备名称转换为某物理设备名称的功能

两大优点：设备分配时的灵活性；易于实现I/O重定向

逻辑设备名称到物理设备名称的转换

逻辑设备表LUT

LUT的设置可采取两种方式：

整个系统设置一张LUT，主要用于单用户

为每个用户设置一张LUT，每当用户登录时，便为该用户建立一个进程，同时也为之建立一张LUT，并将该表放入进程的PCB中。用于多用户

逻辑设备名	物理设备名	驱动程序入口地址	逻辑设备名	系统设备表指针
/dev/tty	3	1024	/dev/tty	3
/dev/printer	5	2046	/dev/printer	5
⋮	⋮	⋮	⋮	

(a)

(b)

与设备无关的软件

功能模块（屏蔽硬件差异，为用户提供一致、可靠的 I/O 支持）

设备驱动程序的统一接口：所有设备驱动程序都遵循统一的接口格式，如

open()、read()、write()、close()，为操作系统或用户层提供统一的设备操作方法，而不需要了解每个设备的底层细节

缓冲管理：在 I/O 速度慢于 CPU 时提供数据缓存区，减少等待，提高系统吞吐率，有单缓冲、双缓冲、循环缓冲等

差错控制：在传输过程中检测并处理 I/O 错误，增强系统健壮性，检测硬件错误码或状态寄存器，重试（暂时性错误）或上报错误（持久性错误）

对独立设备的分配与回收：确保多个进程/程序安全、独立地使用设备，防止设备冲突

独立于设备的逻辑数据块：隐藏不同设备的数据格式和块结构差异，为用户提供统一的逻辑块接口

设备分配

系统在进行设备分配时，应考虑这样几个因素：

设备的固有属性：独占设备、共享设备、虚拟设备

设备分配算法：先来先服务、优先级高者优先

设备分配的安全性：安全分配方式、不安全分配方式（针对死锁条件中的“请求和保持”，前者发出I/O请求后阻塞，cpu与IO顺序工作，后者可发出多个请求）

独占设备的分配程序

基本的设备分配程序

1.分配设备：根据I/O请求中的物理设备名查找系统设备表SDT，从中找出该设备的DCT，再根据DCT上的设备状态字段，得知设备是否在忙，若忙便将请求IO的PCB挂在设备队列上，否则，计算本次设备分配的安全性，若安全则分配，否则，挂在设备等待队列上

2.分配控制器：系统分配设备给进程后，在DCT中找出与该设备连接的控制器COCT，COCT的状态字段反应了控制器是否忙碌，若忙则将PCB挂在控制器的等待队列上，否则，分配控制

器给进程

3.分配通道：在COCT中找到与该控制器相连接的CHCT，根据CHCT状态是否忙碌分配或挂起设备分配程序的改进

增加设备无关性，在分配设备阶段，使用逻辑设备名在SDT上逐个查询可用设备，当所有设备都不可用时才会挂起才设备等待队列上

用户层的I/O软件

系统调用与库函数

用户程序通过调用对应的库函数使用系统调用

库函数的集合也是I/O系统的组成部分

假脱机（SPOOLing）系统

假脱机技术

假脱机是一种通过软件模拟脱机设备、让 CPU 和外围设备操作并行进行的技术，从而缓解 CPU 与 I/O 速度不匹配的问题，提高计算效率

SPOOLing系统的组成

输入井和输出井；输入缓冲区和输出缓冲区；输入进程SPi和输出进程Spo，并管理程序

请求打印队列是由若干张请求打印表所形成的队列，系统为每一个请求打印的进程建立一张请求打印表

特点

提高了I/O的速度

将独占设备改造为共享设备

实现了虚拟设备功能

共享打印机

由输出进程在输出井中为之申请一空闲盘块区，并将要打印的数据送入其中

输出进程再为用户进程申请一张空白的用户请求打印表，并将用户的打印要求填入其中，再将该表挂到请求打印队列上

缓冲区管理

缓冲管理的主要功能是组织好缓冲区，并提供获得和释放缓冲区的手段

缓冲的引入原因

缓和CPU与I/O设备间速度不匹配的矛盾。

减少对CPU的中断频率，放宽对CPU中断响应时间的限制。

提高CPU和I/O设备之间的并行性

单缓冲和双缓冲

单缓冲

单缓冲是OS提供的最简单的一种缓冲形式。每当一个用户进程发出一I/O请求时，操作系统便为主存中为之分配一缓冲区

在块设备输入时，缓冲区用于暂存块设备输入的一块数据；

在字符设备输入时，缓冲区用于暂存用户输入的一行数据。

双缓冲

为了加快输入、输出速度和提高设备利用率，又引入了双缓冲工作方式，也称为缓冲对换方式。在设备输入时，先将数据输入第一缓冲区，装满后便转向第二缓冲区

循环缓冲

多个缓冲区

在循环缓冲中含有多个缓冲区，每个缓冲区的大小相同。缓冲区可分成三种类型：

空缓冲区R 已装满数据的缓冲区G 现行工作缓冲区C

多个指针

Nextg----指示计算进程下一个可用的缓冲区G；

Nexti----指示输入进程下次可用的空缓冲区R；

Current----指示计算进程正在使用的缓冲区单元。

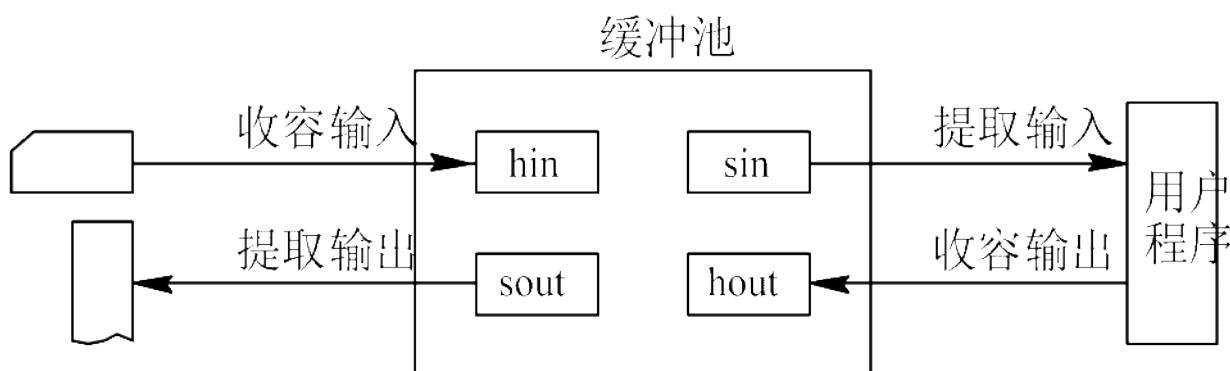
缓冲池

三大队列

空缓冲队列emq。是由空缓冲区所链成的队列。其队首指针F(emq)和队尾指针L(emq)分别指向该队列的首缓冲区和尾缓冲区。

输入队列inq。是由装满输入数据的缓冲区所链成的队列。其队首指针F(inq)和队尾指针L(inq)分别指向该队列的首、尾缓冲区。

输出队列outq。是由装满输出数据的缓冲区所链成的队列。其队首指针F(outq)和队尾指针L(outq)分别指向该队列的首、尾缓冲区。



四种工作方式

收容输入工作方式

在输入进程需要输入数据时，便调用Getbuf(emq)过程，从emq队列的队首摘下一空缓冲区，把它作为收容输入工作缓冲区hin。然后，把数据输入其中，装满后再调用Putbuf(inq, hin)过程，将该缓冲区挂在输入队列inq的队尾。

提取输入工作方式

当计算进程需要输入数据时，调用Getbuf(inq)过程，从输入队列取得一缓冲区作为提取输入工作缓冲区，计算进程从中提取数据。计算进程用完该数据后，再调用Putbuf(emq, sin)过程，将该缓冲挂到空缓冲队列emq上。

收容输出工作方式

当计算进程需要输出时调用Getbuf(emq)过程，从空缓冲队列emq的队首取得一空缓冲，作为收容输出工作缓冲区hout。当其中装满输出数据后，又调用Putbuf(outq, hout)过程，将该缓冲区挂在outq末尾。

提取输出工作方式

当要输出时，由输出进程调用Getbuf(outq)过程，从输出队列的队首取得一装满输出数据的缓冲区，作为提取输出工作缓冲区sout。在数据提取完后，再调用Putbuf(emq, sout)过程，将它挂在空缓冲队列的末尾。

磁盘存储器的性能与调度

数据的组织和格式

磁道：磁盘旋转时, 磁头的轨迹形成一道道同心圆, 信息就存贮在这些同心圆上。我们把这些同心圆叫做磁道。

扇区：磁道划分成的扇段。每个扇区可以存贮 128×2^n 个字节, 其中 $n = 0, 1, 2, 3$ 。

容量：= 面数 $\times$ 磁道数 $\times$ 每道扇区数 $\times$ 每扇区字节数

交错数：相邻编号的扇区会交错地排在磁道上的间隔扇区数, 访问磁盘上一个完整磁道, 磁盘所转的圈数恰好等于交错数。因此也可以将交错数理解为访问一个完整磁道时磁盘所转的圈数。

磁盘访问时间

寻道时间 T_s

把磁臂(磁头)移动到指定磁道上所经历的时间

旋转延迟时间 T_r

指定扇区移动到磁头下面所经历的时间

传输时间 T_t

把数据从磁盘读出或向磁盘写入数据所经历的时间

一次访盘时间 = 寻道时间 + 旋转延迟时间 + 存取时间

磁盘调度算法

减少寻道时间、减少延迟时间

先来先服务 FCFS (公平简单但是平均寻道时间较长)

最短寻道时间优先 SSTF (因为局部性原理会导致其他进程得不到响应)

扫描 SCAN 算法：磁头沿着一个方向 (如从外往里) 移动时, 按顺序处理所有该方向上的请求; 到达磁盘一端后, 磁头调头反向移动, 继续处理反方向上的请求

循环扫描算法 CSCAN：磁头只沿一个方向 (例如从低号磁道到高号磁道) 扫描磁盘请求; 到达磁盘的一端后, 磁头不处理回程的请求, 而是直接跳回另一端, 重新开始新一轮扫描; 整个扫描过程呈现为一个单方向的循环

N步扫描 N-Step_SCAN 算法：将磁盘请求队列分成若干个长度为 N 的子队列, 磁盘调度将按 FCFS 算法依次处理这些子队列。而每处理一个队列时又是按 SCAN 算法, 对一个队列处理完后, 再处理其他队列。当正在处理某子队列时, 如果又出现新的磁盘 I/O 请求, 便将新请求进程放入其他队列, 这样就可避免出现粘着现象

文件和文件系统

数据项、记录 and 文件

数据项是最低级的数据组织形式，可以分为两种类型

基本数据项。这是用于描述一个对象的某种属性的字符集，是数据组织中可以命名的最小逻辑数据单位，即原子数据，又称为数据元素或字段。它的命名往往与其属性一致。

组合数据项。它是由若干个基本数据项组成的，简称组项。

记录是一组相关数据项的集合，用于描述一个对象在某方面的属性

关键字是能唯一标识一个记录的数据项

文件是指由创建者所定义的、具有文件名的一组相关信息的集合，可分为有结构文件和无结构文件两种

在有结构的文件中，文件由若干个相关记录组成；而无结构文件则被看成是一个字符流。文件在文件系统中是一个最大的数据单位，它描述了一个对象集

文件具有自己的属性，可以包括：类型、长度、物理位置、建立时间

文件类型

按用途分类	按文件中的数据形式分类	按存取控制属性分类	按文件的逻辑结构分类	按文件的物理结构分类
系统文件	源文件	只执行文件	有结构文件	顺序文件
用户文件	目标文件	只读文件		链接文件
库文件	可执行文件	读写文件	无结构文件	索引文件

文件系统的层次结构

文件系统接口	
对对象操纵和管理的软件集合	逻辑文件系统
	基本I/O管理程序（文件组织模块）
	基本文件系统（物理I/O层）
	I/O控制层（设备驱动程序）
对象及其属性说明	

中间层核心功能：管理文件存储空间、文件目录，将文件的逻辑地址转化为物理地址，对文件读和写的管理和对文件的共享和保护

最底层功能：管理文件、目录、磁盘存储空间

最高层功能：提供命令接口（终端）和程序接口（系统调用）

文件操作

创建、删除、读写、设置（起始）读写位置

打开文件：文件的属性(包括该文件在外存上的物理位置)从外存拷贝到内存打开文件表的一个表目中，并将该表目的编号(或称为索引)返回给用户（避免了对该文件的再次检索。这样不仅节省了大量的检索开销，也

显著地提高了对文件的操作速度)

关闭文件：利用“关闭”(close)系统调用来关闭此文件，OS将会把该文件从打开文件表中的表目上删除掉

其他操作：对文件属性更改、对目录增删

文件的逻辑结构

按文件是否有结构分类

有结构文件：即记录式文件。大量的数据结构和数据库采用此类文件。在此类文件中，记录的长度可分定长和不定长两类

无结构文件：即流式文件。大量的源程序、可执行文件、库函数等采用的是此类文件。

按文件的组织方式分类

顺序文件：指由一系列记录按某种顺序排列所形成的文件，其中的记录可以是定长记录或可变长记录

索引文件：指为可变长记录文件建立一张索引表，为每个记录设置一个表项，以加速对记录检索的速度

索引顺序文件：这是顺序文件和索引文件相结合的产物，这里，在为每个文件建立一张索引表时，并不为每个记录建立一个索引表项，而是为一组记录中的第一个记录建立一个索引表项

顺序文件

优点：对诸记录进行批量存取时，存取效率最高；只有顺序文件才能存储在磁带上并能有效地工作

缺点：交互应用时性能很差；增加或修改一个记录比较困难

记录寻址

隐式寻址：定长记录，已知当前记录的逻辑地址，便很容易确定下一个记录的逻辑地址

显式寻址：通过文件中记录的位置。此时，在文件中的每一个记录，可用从0到N-1的整数来标识，即用一个整数来唯一地标识一个记录；利用关键字。此时用户必须指定一个字段作为一个关键字，通过指定的关键字来查找记录

文件目录

文件控制块

文件与文件控制块一一对应，而把文件控制块的有序集合称为文件目录。换言之，一个文件控制块就是一个文件目录项。通常一个文件目录也被看作一个文件，称目录文件

基本信息	① 文件物理位置	● 存放文件的设备名；
		● 文件在外存上的盘块号；
	② 文件名	● 指示文件所占用磁盘块数或字节数的文件长度
存储控制信息	① 文件主的存取权限	
	② 核准用户的存取权限	
	③ 一般用户的存取权限	
使用信息	① 文件的建立日期和时间	
	② 文件上一次修改的日期和时间	
	③ 当前使用信息	

索引结点

在检索目录文件的过程中，只用到了文件名，一概用不着该文件的描述信息。仅当找到一个其文件名与指定文件名相匹配的目录项时，才需从该目录项中读出该文件的物理地址

磁盘索引结点

它是指存放在磁盘上的索引结点。每个文件有唯一的一个磁盘索引结点

内存索引结点

指存放在内存的索引结点。当文件被打开时，要将磁盘索引结点拷贝到内存索引结点中，以便于以后使用

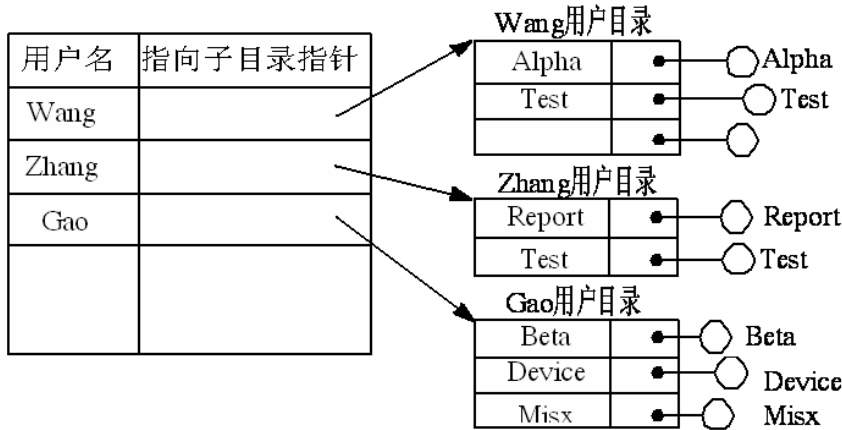
单级文件目录

单级目录（Single Level Director）结构是最简单的目录结构。在整个文件系统中只建立一张目录表，每个文件占一个目录项。目录项中含文件名、文件扩展名、文件长度、文件类型、文件物理地址以及其它文件属性。此外，为表明每个目录项是否空闲，又设置了一个状态位

优点	简单，能实现目录管理的基本功能----按名存取
缺点	🔴 查找速度慢；
	🟡 不允许重名；
	🟢 不便于实现文件共享。

两级文件目录

为了克服单级目录结构的缺点，可为每用户建立一个单独的用户文件目录UFD，它由用户所有文件的文件控制块组成。再在系统中建立一个主文件目录MFD。在主文件目录中，每个用户文件目录都占有一个目录项；其目录项中包括用户名和指向该用户目录文件的指针



🔴 提高了检索目录的速度；
🟡 在不同的用户目录中，可使用相同的文件名，只要在用户自己的UFD中其文件名都是唯一的；
🟢 不同用户可使用不同文件名来访问系统中的同一个共享文件

目录查询技术：线性检索法和Hash方法

Hash方法

在利用Hash法索引查找目录时，如果目录表中相应的目录项是空的，则表示系统中并无指定文件。如果目录项中的文件名与指定文件名相匹配，则表示该目录项正是所要寻找的文件所对应的目录项，故可从中找到该文件所在的物理地址。

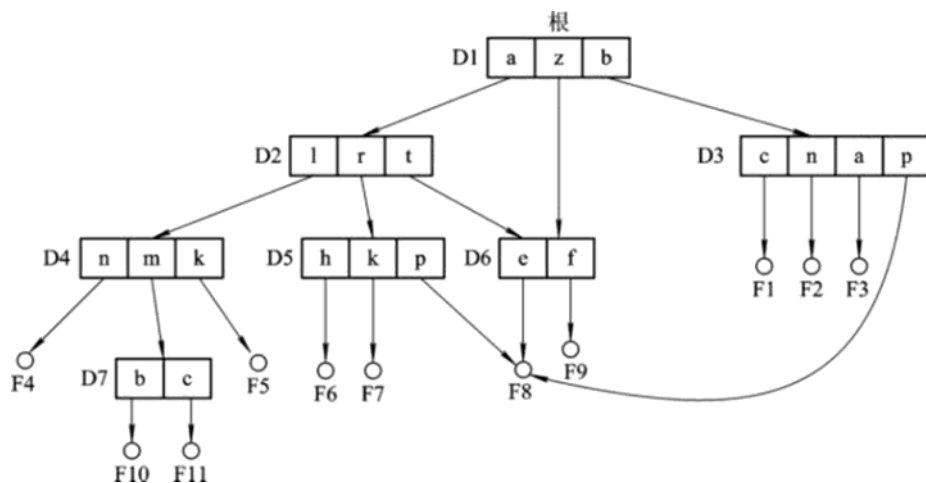
如果在目录表的相应目录项中的文件名与指定文件名并不匹配，则表示发生了“冲突”，此时须将其Hash值再加上一个常数(该常数应与目录的长度值互质)，形成新的索引值，再返回到第一步重新开始查找。

文件共享

基于有向无循环图实现文件共享

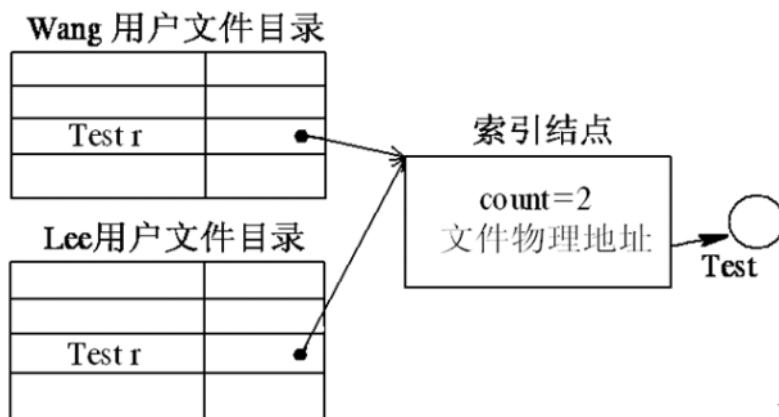
允许一个文件可以有多个父目录，即有多个属于不同用户的多个目录，同时指向同一个文件，这样虽会破坏树的特性，但这些用户可用对称的方式实现文件共享，而不必再通过其属主目录来访问

但是如果数据节点被更改会导致无法同步



利用索引结点

诸如文件的物理地址及其它的文件属性等信息，不再是放在目录项中，而是放在索引结点中。在文件目录中只设置文件名及指向相应索引结点的指针，如下图所示



利用符号链实现文件共享

利用符号链接实现文件共享的基本思想，是允许一个文件或子目录有多个父目录，但其中仅有一个作为主(属主)父目录，其它的几个父目录都是通过符号链接方式与之相链接的(简称链接父目录)

在利用符号链方式实现文件共享时，只是文件主才拥有指向其索引结点的指针；而共享该文件的其他用户则只有该文件的路径名，并不拥有指向其索引结点的指针

这样，也就不会发生在文件主删除一共享文件后留下一悬空指针的情况。当文件的拥有者把一个共享文件删除后，如果其他用户又试图通过符号链去访问一个已被删除的共享文件，则会因系统找不到该文件而使访问失败，于是再将符号链删除，此时不会产生任何影响

问题：当其他用户去读共享文件时，系统是根据给定的文件路径名逐个分量(名)地去查找目录，直至找到该文件的索引结点。因此，在每次访问共享文件时，都可能要多次地读盘。这使每次访问文件的开销甚大，且增加了启动磁盘的频率。此外，要为每个共享用户建立一条符号链，而由于链本身实际上是一个文件，尽管该文件非常简单，却仍要为其配置一个索引结点，这也要耗费一定的磁盘空间。

文件保护

影响安全性三大因素：人为、系统、自然

对应的措施

通过存取控制机制，防止由人为因素所造成的文件不安全性。

采取系统容错技术，防止系统部分的故障所造成的文件的不安全性。

建立后备系统，防止由自然因素所造成的不安全性。

保护域

把一个进程能对某对象执行操作的权力，称为访问权

保护域：“域”是进程对一组对象访问权的集合，进程只能在指定域内执行操作。这样，“域”也就规定了

进程所能访问的对象和能执行的操作

进程和域间的静态联系

在进程的整个生命期中，其可用资源是固定的，我们把这种域称为“静态域”。在这种情况下，进程运行的全过程都是受限于同一个域，这将会使赋予进程的访问权超过了实际需要

进程和域间的动态联系

在进程和域之间，也可以是一对多的关系，即一个进程可以联系着多个域。在此情况下，可将进程的运行分为若干个阶段，其每个阶段联系着一个域，这样便可根据运行的实际需要来规定在进程运行的每个阶段中所能访问的对象

访问矩阵

基本的访问矩阵

域 \ 对象	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	Printer 1	Plotter 2
D ₁	R	R, W						
D ₂			R	R, W, E	R, W		W	
D ₃						R, W, E	W	W

具有域切换权的访问矩阵

域 \ 对象	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	Printer 1	Plotter 2	域 D ₁	域 D ₂	域 D ₃
域 D ₁	R	R, W								S	
域 D ₂			R	R, W, E	R, W		W				S
域 D ₃						R, W, E	W	W			

拷贝权

我们可利用拷贝权将在某个域中所拥有的访问权(access(i, j))扩展到同一列的其它域中，亦即，为进程在其它的域中也赋予对同一对象的访问权(access(k, j))，如下图所示

域 \ 对象	F ₁	F ₂	F ₃
D ₁	E		W*
D ₂	E	R*	E
D ₃	E		

(a)

域 \ 对象	F ₁	F ₂	F ₃
D ₁	E		W*
D ₂	E	R*	E
D ₃	E	R	W

(b)

所有权

人们不仅要求能将已有的访问权进行有控制的扩散，而且同样需要能增加某种访问权，或者能删除某种访问权，此时，可利用所有权(O)来实现这些操作。

域 \ 对象	F ₁	F ₂	F ₃
D ₁	O, E		W
D ₂		R*, O	R*, O, W
D ₃	E		

(a)

域 \ 对象	F ₁	F ₂	F ₃
D ₁	O, E		
D ₂		O, R*, W*	R*, O, W
D ₃		W	W

(b)

控制权

拷贝权和所有权都是用于改变矩阵内同一列的各项访问权的，或者说，是用于改变在不同域中运行的进程对同一对象的访问权的。控制权则可用于改变矩阵内同一行中(域中)的各项访问权，亦即，用于改变在某个域中运行的进程对不同对象的访问权的。如果在access(i, j)中包含了控制权，则在域D_i中运行的进程可以删除在域D_j中运行的进程对各对象的任何访问权

域 \ 对象	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	Printer 1	Plotter 2	域 D ₁	域 D ₂	域 D ₃
域 D ₁	R	R, W									
域 D ₂			R	R, W, E	R, W		W				Control
域 D ₃						R, E	W	W			

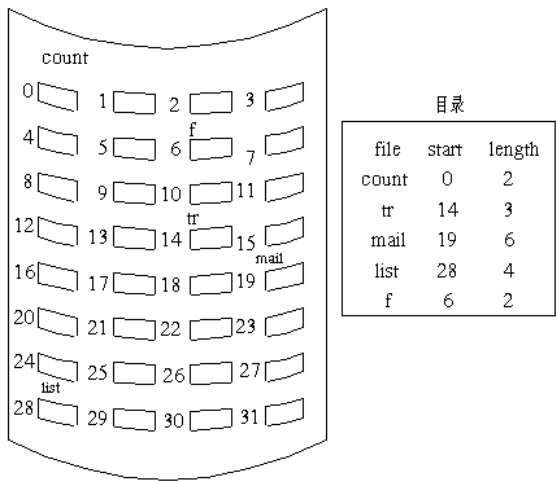
外存的组织方式

空间利用率和访问速率

外存组织方式有：

连续分配

连续组织方式也称连续分配方式。要求为每一个文件分配一组相邻接的盘块。一组盘块的地址定义了磁盘上的一段线性地址。在采用连续分配方式时，可把逻辑文件中的记录，顺序地存取到邻接的各物理盘块中，这样形成的物理文件称顺序文件



连续分配的主要优点如下：

- (1) 顺序访问容易
- (2) 顺序访问速度快

连续分配的主要缺点如下：

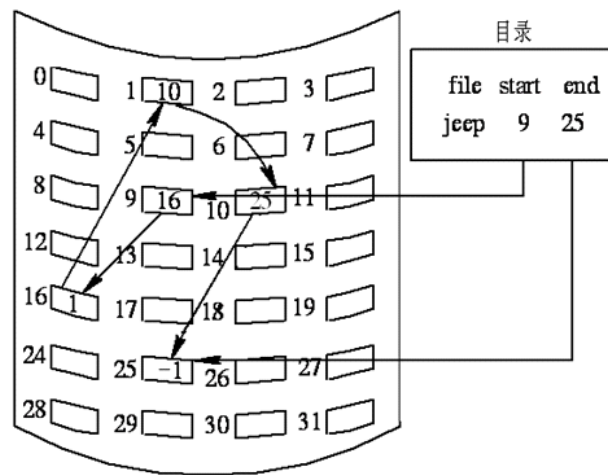
- (1) 要求为一个文件分配连续的存储空间（会产生碎片）
- (2) 必须事先知道文件的长度（文件长度只能估算）
- (3) 不能灵活地删除和插入记录
- (4) 对于那些动态增长的文件很难分配空间

链接分配

在采用链接分配方式时，可通过在每个盘块上的链接指针，将同属于一个文件的多个离散的盘块链接成一个链表，由次所形成的物理文件称为链接文件。链接方式又可分为隐式链接和显式链接

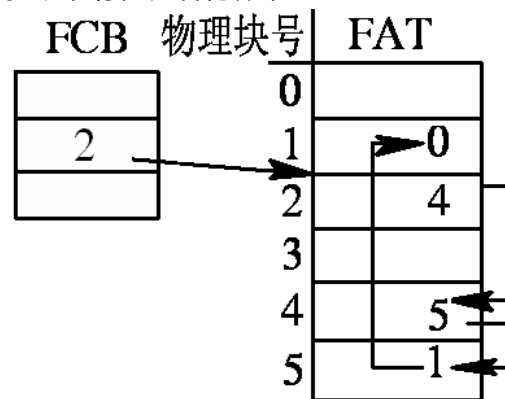
隐式链接

在采用隐式链接分配方式时，在文件目录的每个目录项中，都须含有指向链接文件第一个盘块和最后一个盘块的指针
缺点是可靠性差，随机访问效率低



显式链接

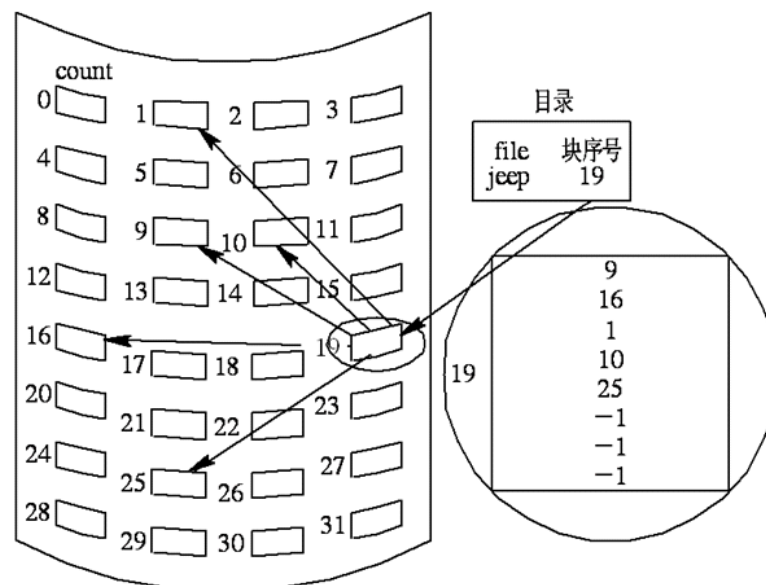
指把用于链接文件各物理块的指针，显式地存放在内存的一张链接表中。该表是整个磁盘仅设置一张。由于分配给文件的所有盘块号都放在该表中，故把该表称为文件分配表FAT。



索引分配

单级索引组织方式

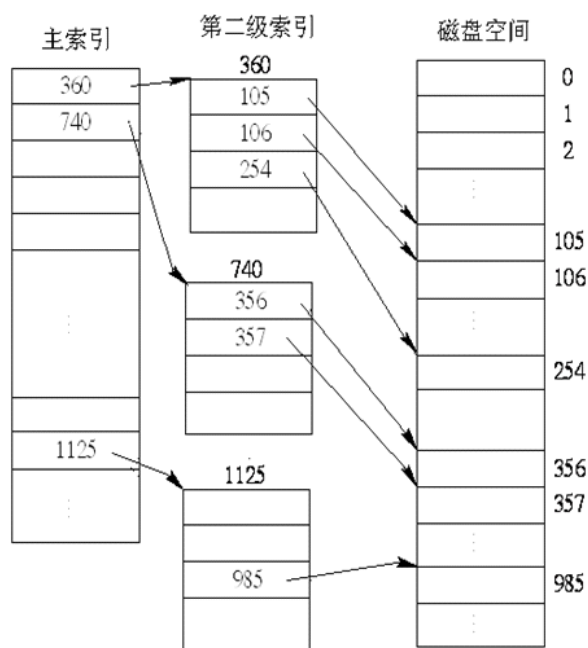
索引分配方法为每个文件分配一个索引块（表），把分配给该文件的所有盘块号，都记录在该索引表中，因而该索引块就是一个含有许多盘块号的数组（花费内存较多）



多级索引分配

当OS为一个大文件分配磁盘空间时，可能出现索引块太多的情况，这时是低效的。应 为这些索引块在建立一级索引，称第一级索引，这样便形成了两级

索引分配方式。文件非常大时，还可用三级、四级索引分配方式



混合索引分配方式

是指将多种分配方式相结合而形成的一种分配方式。它们把所有的地址项分成两类，即直接地址和间接地址

(1) 直接地址

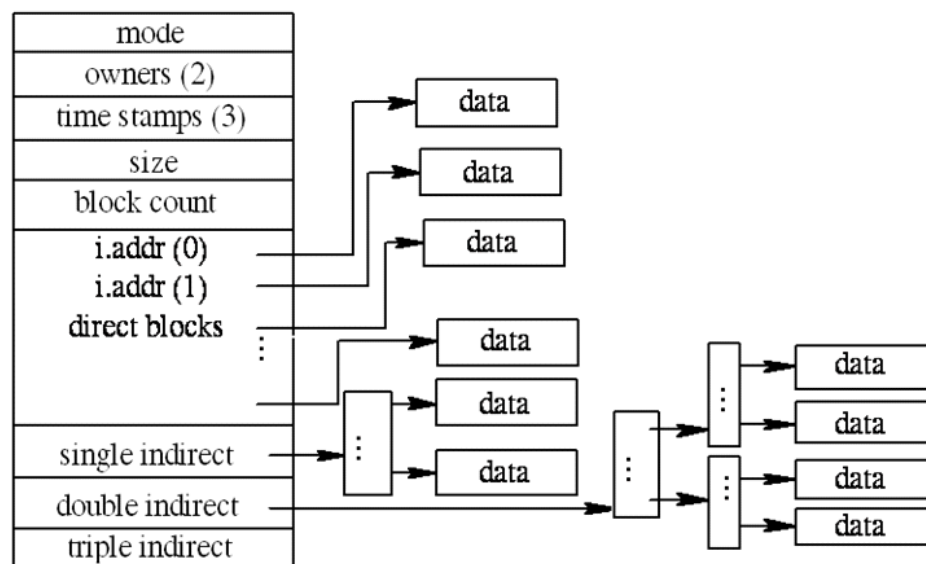
为了提高对文件的检索速度，在索引结点中可设置10个直接地址项，即用iaddr(0)-iaddr(9)来存放直接地址。换言之，在这里的每项中所存放的是该文件数据的盘块的盘块号。假如每个盘块的大小为4KB，当文件不大于40KB时，便可直接从索引结点中读出该文件的全部盘块号。

(2) 一次间接地址

对于大、中型文件，只采用直接地址是不现实的。为此，可再利用索引结点中的地址项iaddr(10)来提供一次间接地址。这种方式的实质就是一级索引分配方式。图中的一次间址块也就是索引块，系统将分配给文件的多个盘块号记入其中。在一次间址块中可存放1K个盘块号，因而允许文件长达4MB。

(3) 多次间接地址

当文件长度大于4MB+40KB时(一次间址与10个直接地址项)，系统还须采用二次间址分配方式。这时，用地址项iaddr(11)提供二次间接地址。该方式的实质是两级索引分配方式。系统此时是在二次间址块中记入所有一次间址块的盘号。在采用二次间址方式时，文件最大长度可达4GB。同理，地址项iaddr(12)作为三次间接地址，其所允许的文件最大长度可达4TB。



文件存储空间的管理

空闲表法

空闲表法属于连续分配方式。它为每个文件分配一个连续的存储空间。系统为外存上的所有空闲区建立一张空闲表，每个空闲区对应于一个空闲表项

空闲盘区的分配与内存的动态分配类似，同样是采用首次适应算法、循环首次适应算法等。例如，在系统为某新创建的文件分配空闲盘块时，先顺序地检索空闲表的各表项，直至找到第一个其大小能满足要求的空闲区，再将该盘区分配给用户(进程)，同时修改空闲表。系统在对用户所释放的存储空间进行回收时，也采取类似于内存回收的方法，即要考虑回收区是否与空闲表中插入点的前区和后区相邻接，对相邻接者应予以合并。

序号	第一空闲盘块号	空闲盘块数
1	2	4
2	9	3
3	15	5
4	-	-

空闲链表法

(1) 空闲盘块链

它是将磁盘上的所有空闲存储空间，以盘块为基本元素拉成一条链。优点是用于分配和回收一个盘块的过程非常简单；缺点是空闲盘块链可能很长。

(2) 空闲盘区链

这是将磁盘上的所有空闲盘区（每个盘区可包含若干个盘块）拉成一条链。在每个盘区上除了含有用于指示下一个空闲盘区的指针外，还应标有指明本盘区大小（盘块数）的信息。这种方法分配和回收过程较复杂，但空闲盘区链较短。

位示图法

位示图是利用二进制的一位来表示磁盘中一个盘块的使用情况。当其值为“0”时，表示对应的盘块空闲；为“1”时表示已分配。由所有盘块对应的位构成一个集合，称为位示图。位示图也可描述为一个二维数组map: Var map:array[1...m,1...n] of bit;

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	1	1	0	0	0	1	1	1	0	0	1	0	1	1	1	0
2	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	1
3	1	1	1	0	0	0	1	1	1	1	1	1	0	0	0	0

盘块的分配

根据位示图进行盘块分配时，可分三步进行：

- 1 顺序扫描位示图，从中找出一个或一组值均为“0”的二进制位
- 2 将找到的二进制位，转换成与之相应的盘块号
- 3 修改位示图，令 $\text{map}[i,j]=1$

盘块的回收

盘块的回收分两步：

- 1 将回收盘块的盘块号转换成位于图中的行号和列号。转换公式为：
$$i=(b-1)\text{DIV}n+1$$
$$j=(b-1)\text{MOD}n+1$$
- 2 修改位示图。令 $\text{map}[i,j]=0$ 。

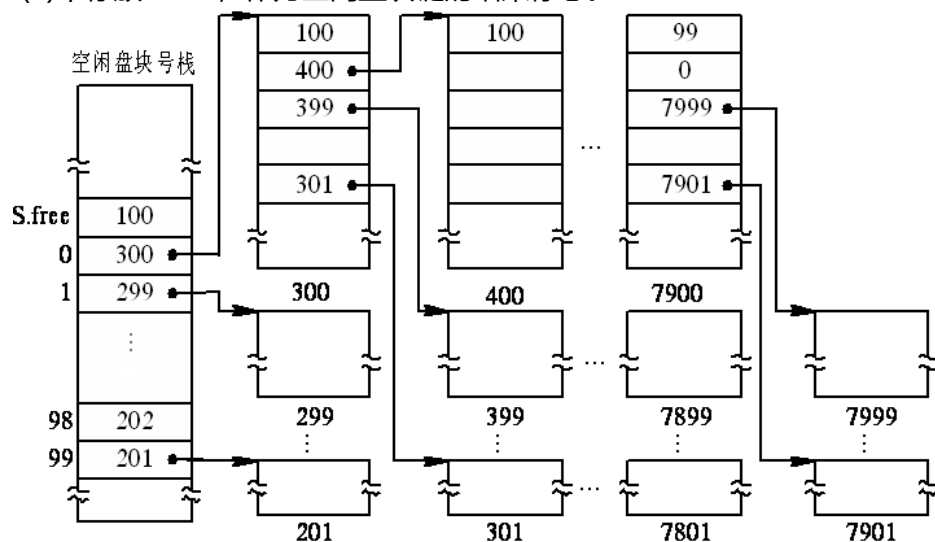
成组链接法

文件区中的所有空闲盘块，被分成若干个组。

将每一组含有的盘块总数N和该组所有的盘块号，记入其前一组的第一个盘块的 $S.\text{free}(0)\sim S.\text{free}(99)$ 中。

将第一组的盘块总数和所有的盘块号，记入空闲盘块号栈中。

最末一组只有99个盘块，盘块号记入其前一组第一盘块的 $S.\text{free}(1)\sim S.\text{free}(99)$ 中。而在 $S.\text{free}(0)$ 中存放“0”，作为空闲盘块链的结束标志。



提高磁盘I/O速度的途径

磁盘高速缓存

数据交付方式

数据交付就是指将磁盘高速缓存中的数据传送给请求者进程。系统可以采取两种方式将数据交付给请求进程：

- (1) 数据交付 (形参)
- (2) 指针交付 (实参)

置换算法

系统在设计其高速缓存的置换算法时，除了考虑到最近最久未使用这一原则外，还考虑了以下几点：

- (1) 访问频率（优先替换访问频率低的）
- (2) 可预见性（替换很久不会访问的）
- (3) 数据的一致性（优先替换未修改的）

周期性地写回磁盘

提高磁盘I/O速度的其它方法

能有效地提高磁盘I/O速度的方法还有许多，如提前读、延迟写、优化物理块的分布、虚拟盘(RAM)

廉价磁盘冗余阵列RAID

并行交叉存取:有多台磁盘驱动器，系统将每一盘块中的数据分为若干个子盘块数据，再把每一个子盘块的数据分别存储到各个不同磁盘中的相同位置上。以后当要将一个盘块的数据传送到内存时，采取并行传输方式，将各个盘块中的子盘块数据同时向内存中传输，从而使传输时间大大减少。

RAID的分级

RAID的优点

- (1) 可靠性高，除了RAID 0级外，其余各级都采用了容错技术。当阵列中某一磁盘损坏时，并不会造成数据的丢失。此时可根据其它未损坏磁盘中的信息来恢复已损坏的盘中的信息。其可靠性比单台磁盘机高出一个数量级。
- (2) 磁盘I/O速度高，由于采取了并行交叉存取方式，可使磁盘I/O速度提高N-1倍。
- (3) 性能/价格比高，RAID的体积与具有相同容量和速度的大型磁盘系统相比，只是后者的1/3，价格也只是后者的1/3，且可靠性高。换言之，它仅以牺牲1/N的容量为代价，换取了高可靠性。

提高磁盘可靠性的技术(系统层面防止问题)

第一级容错技术SFT- I：是最基本的一种磁盘容错技术，主要用于防止因磁盘表面缺陷所造成的数据丢失。它包含双份目录、双份文件分配表及热修复重定向和写后读校验等措施。

第二级容错技术SFT- II：

磁盘镜像

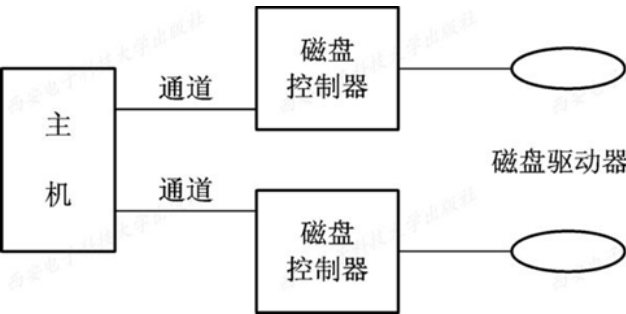
为了避免磁盘驱动器/控制器发生故障而丢失数据，便增设了磁盘镜像功能。为实现该功能，须在同一磁盘控制器下，再增设一个完全相同的磁盘驱动器



磁盘双工

如果控制这两台磁盘驱动器的磁盘控制器发生故障，或主机到磁盘控制器之间的通道发生故障，磁盘镜像功能便起不到数据保护的作用。因此，在第二级容错技术

中，又增加了磁盘双工功能，即将两台磁盘驱动器分别接到两个磁盘控制器上，同样使这两台磁盘机镜像成对



基于集群技术的容错功能

模式	结构	特点	适用场景
双机热备份	主-备	简单稳定，切换快，资源利用率低	核心业务系统、高可用数据库
双机互为备份	主-主	利用率高，双向接管，管理复杂	多任务服务器、容错 Web 服务
公用磁盘模式	多机+共享磁盘	无需数据复制，动态切换，依赖存储系统	文件服务、虚拟机集群等

后备系统

设备/概念	用途	主要特点
后备系统	数据恢复、灾难恢复	可由磁带、硬盘等构成，强调冗余安全
磁带机	长期数据归档	成本低、速度慢、容量大
硬盘	主存储 & 备份存储	快速、随机访问、普遍使用
光盘驱动器	便携式/只读数据分发	便宜但容量小，逐渐被淘汰

数据一致性控制

事务：事务是用于访问和修改各种数据项的一个程序单位。事务也可以被看做是一系列相关读和写操作

恢复算法

由于一组被事务Ti修改的数据以及它们被修改前和修改后的值都能在事务记录表中找到，因此，利用事务记录表系统能处理任何故障而不致使故障造成非易失性存储器中信息的丢失。恢复算法可利用以下两个过程：

- (1) undo 〈Ti〉。该过程把所有被事务Ti修改过的数据恢复为修改前的值。
- (2) redo 〈Ti〉。该过程能把所有被事务Ti修改过的数据设置为新值。

检查点

引入检查点后，可以大大减少恢复处理的开销。因为在发生故障后，并不需要对事务记录表中的所有事务记录进行处理，而只需对最后一个检查点之后的事务记录进行处理。因此，恢复例程首先查找事务记录表，确定在最近检查点以前开始执行的最后的事务

Ti。在找到这样的事务后，再返回去搜索事务记录表，便可找到第一个检查点记录，恢复例程便从该检查点开始返回搜索各个事务的记录，并利用redo和undo过程对它们进行处理。

并发控制

利用互斥锁和共享锁实现顺序性

共享锁与互斥锁的区别在于：互斥锁仅允许一个事务对相应对象执行读或写操作，而共享锁则允许多个事务对相应对象执行读操作，但不允许其中任何一个事务对对象执行写操作

重复数据的数据一致性问题

重复文件的一致性

链接数一致性检查